

PSIR PROJEKT

Kacper Moździerz

20 marca 2026

Spis treści

1	Wstęp	2
1.1	Cel i architektura systemu	2
1.2	Funkcjonalność serwera	2
1.3	Funkcjonalność węzła IoT	2
1.4	Wymagane rezultaty	3
2	Rozwiązanie	3
2.1	Wykorzystywane narzędzia	3
2.2	Prezentacja wyniku końcowego	4
2.3	Zaprojektowanie i specyfikacja protokołu warstwy aplikacyjnej (ALP)	14
2.3.1	Struktura pakietu	14
2.3.2	Typy wiadomości i logika sterowania	14
2.3.3	Rejestracja węzła	14
2.3.4	Dystrybucja danych (String L-systemu)	15
2.3.5	Sterowanie pracą (Renderowanie)	15
2.3.6	Zarządzanie stanem (Handover)	15
2.3.7	Szczegóły struktur danych (Payloads)	15
2.3.8	Parametry sieciowe	15
2.4	Serwer	16
2.4.1	Konfiguracja sieciowa serwera i konfiguracja maszyny serwera	16
2.4.2	server_defs.h – architektura danych i konfiguracja	20
2.4.3	main.c – funkcja main, koordynacja w systemie	20
2.4.4	net_core.c – obsługa komunikacji sieciowej i handover'u	20
2.4.5	lsystem.c - opis znajduje się w dalszej części raportu 2.6	21
2.4.6	utils.c – przetwarzanie danych wyjściowych i obsługa pikseli	21
2.4.7	globals.c – definicje zasobów współdzielonych	21
2.5	Węzeł IoT	22
2.6	L-system – logika generowania i interpretacji geometrycznej fraktali	27
2.6.1	Dostępne L-systemy	28
2.6.2	Opis pojedynczego L-systemu w pliku .txt - roślina, plant.txt	35
3	Implementacja Mechanizmu Rozproszonego	36
3.1	Podział Przestrzeni Roboczej	36
3.2	Mechanizm Handover (Przekazywanie Stanu)	36
3.3	Niezawodność Transmisji	36
3.4	Arytmetyka Stałoprzecinkowa	37
4	Architektura Oprogramowania	37
4.1	Serwer	37
4.2	Szczegóły implementacji Węzła (Klient)	38
4.2.1	Maszyna Stanów	38
4.2.2	Interpretacja Grafiki Żółwia (Turtle Graphics)	38
4.3	Konfiguracja środowiskowa (Pliki infile)	38

1 Wstęp

Projekt systemu Lindenmayera polega na stworzeniu rozproszonej aplikacji, która wyrenderuje słowo wygenerowane z systemu L-systemu o zadanej długości wyprowadzenia.

1.1 Cel i architektura systemu

- Zbudowanie aplikacji rozproszonej, czyli stworzenie aplikacji zdolnej do rozproszonego renderowania słowa (ciągu znaków) wygenerowanego z systemu Lindenmayera (L-system).
- Renderowanie ASCII art, czyli grafika wyjściowa powinna być wygenerowana jako ASCII Art (np. jeden znak = jeden piksel, z możliwością eksperymentowania z wyższą rozdzielczością).
- Podział ról, czyli system musi składać się z dwóch głównych elementów: serwer (jednostka centralna, koordynująca) i węzły IoT (klienty, wykonujące renderowanie w swoich podobszarach).
- Implementacja serwera, czyli serwer musi być zaimplementowany w języku C, używając wyłącznie API systemu Linux (bez bibliotek, platform czy middleware).
- Implementacja węzłów IoT, czyli węzły muszą być zaimplementowane na emulatorze Arduino z wykorzystaniem standardowego API Arduino i rozszerzeń PSIR. Emulator obsługuje tylko UDP.
- Stworzenie protokołu ALP, czyli opracowanie własnego, binarnego protokołu warstwy aplikacyjnej (ALP) do komunikacji między klientem a serwerem. Protokół powinien działać na protokole UDP i zawierać prosty schemat niezawodności (np. retransmisja po braku ACK).

1.2 Funkcjonalność serwera

Serwer powinien realizować następujące zadania:

- Wczytanie L-systemu i długości wyprowadzenia (z wiersza poleceń lub pliku).
- Akceptowanie rejestracji od węzłów IoT.
- Przypisanie podobszaru do każdego zarejestrowanego węzła.
- Przekazanie węzłom niezbędnych informacji do renderowania
- Rozpoczęcie renderowania.
- Obsługa przekazania (handover).
- Zbieranie fragmentów wygenerowanego obrazu.
- Wyświetlanie całościowego obrazu (ASCII Art).
- Aktualizowanie metryk i generowanie komunikatów diagnostycznych.

1.3 Funkcjonalność węzła IoT

Węzeł IoT powinien realizować następujące zadania:

- Rejestracja u serwera.
- Odbiór przypisanego podobszaru i danych od serwera.
- Przeprowadzanie renderowania.
- Wykrywanie przekazania (handover) i raportowanie go do serwera.
- Wykrywanie końca renderowania.
- Przesłanie (upload) serwerowi fragmentu wygenerowanego obrazu.
- Węzeł wybrany jako początkowy powinien odczytać współrzędne początku interpretacji żółwia z pliku środowiskowego.

1.4 Wymagane rezultaty

W ramach projektu należy dostarczyć:

- Kod źródłowy
- Raport zawierający:
 - Specyfikację protokołu ALP (formaty wiadomości, kodowanie binarne, diagramy sekwencji, opis schematu niezawodności).
 - Opis implementacji serwera (komponenty, struktury danych).
 - Opis implementacji węzła (w tym opis plików środowiskowych).
 - Uzyskane obrazy (ASCII art) dla kilku L-systemów.
- Demonstrację działania systemu, która polega na:
 - Uruchomieniu serwera, wprowadzeniu L-systemu i liczby iteracji.
 - Uruchomieniu kilku węzłów.
 - Pozwoleniu systemowi na pracę i wydrukowaniu końcowej figury (ASCII Art) dla co najmniej dwóch L-systemów.

Termin oddania to 12 stycznia 2025, godzina 12.00.

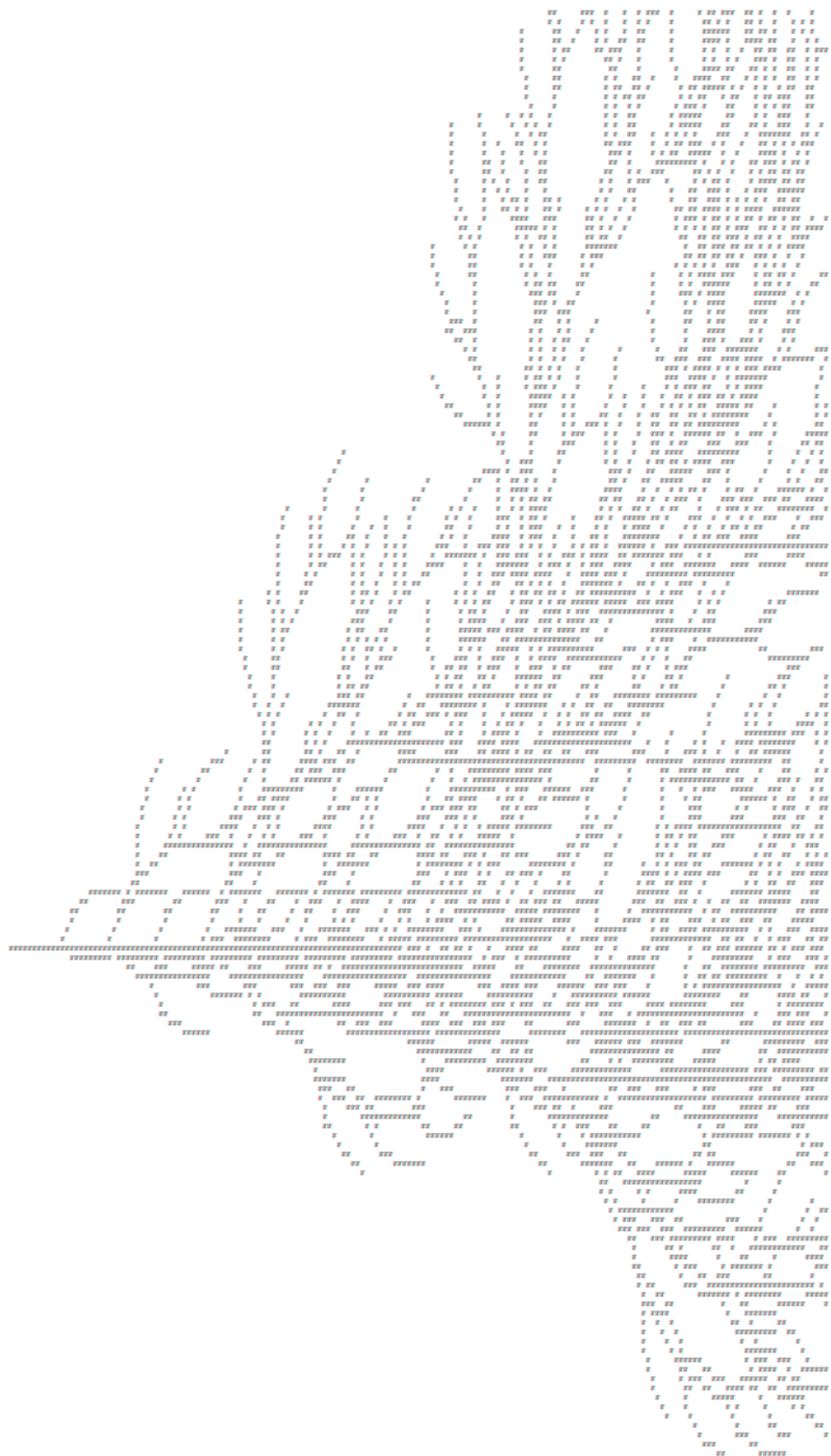
2 Rozwiązanie

2.1 Wykorzystywane narzędzia

- gcc (Debian 12.2.0-14) 12.2.0 - kompilator języka c wbudowany w używane maszyny wirtualne
- Debian GNU/Linux 12 (bookworm) - system maszyny wirtualnej (ta sama co na laboratoriach)
- VirtualBox 7.2.4 r170995 (Qt6.8.0 on windows) - program do wirtualizacji
- MobaXterm Personal Edition v25.3 Build 5384 - program podobny do Putty do pracy z maszynami wirtualnymi (poprzez ssh) dający wygodę, czytelność i przede wszystkim łatwość przesyłu plików między hostem a maszynami

2.2 Prezentacja wyniku końcowego

W prezentacji plikiem z instrukcjami L-systemu będzie plik plant.txt, a wynik działania programu powinien być taki (otwarto w programie Notepad.exe):



Rysunek 1: Docelowy wynik rysowania z pliku plant.txt - widok z notatnika

Widać tu znacznie większą gęstość pikseli fraktala po prawej stronie, co w połączeniu z podziałem pracy węzłów na 4 regiony (każdy węzeł ma własną, równą część osi x) i początkiem ruchu żółwia po skrajnej lewej stronie, sprawi, że pierwszy węzeł pozostanie w dużej mierze bezczynny, a ostatni węzeł będzie miał najwięcej pracy, co będzie widoczne w logach w dalszej części raportu. W sekcji L-system [2.6](#) raportu można zobaczyć inne uzyskane L-systemy w projekcie.

UWAGA: Dokładne informacje o projekcie, w tym struktury katalogów, konfiguracje sieciowe, opisy zawartości plików na maszynach węzłów i serwera, wyniki innych plików txt znajdują się w dalszych podsekcjach, w tej podsekcji jest jedynie wynik ostateczny z opisem kolejnych kroków, które pozwalają doprowadzić do takiego wyniku na podstawie posiadanych plików. Mimo to nie uwzględniono tutaj konfiguracji sieciowej i konfiguracji maszyn wirtualnych, których opisy znajdują się dalej w raporcie, a które mogą okazać się niezbędne do osiągnięcia prezentowanego tu wyniku końcowego.

Na maszynie serwera, będąc w katalogu `l_system_server`, należy skompilować za pomocą polecenia `gcc main.c lsystem.c utils.c net_core.c globals.c -o server_new -lm` wszystkie znajdujące się tam pliki `.c` aby uzyskać plik wynikowy, nazwany tutaj jako `server_new`, który następnie można wykonać:

```
student@iot:~$ cd l_system_server/
student@iot:~/l_system_server$ ./server_new
Serwer nasluchuje na porcie 8080...
Podaj nazwe pliku konfiguracyjnego (np. sierpinski.txt): plant.txt
Wczytano konfiguracje z plant.txt:
-> Iteracje: 4
-> Start Angle: 0.00
-> Angle: 22.00
-> Step: 5.00
-> Draw G: tak
-> Reguly: 1
Wygenerowano ciag: 11116 znakow
Oczekiwanie na wezly (Ctrl+C aby przerwac)...
```

Rysunek 2: Uruchomienie programu na serwerze za pomocą polecenia `./server_new` z wyborem pliku tekstowego `lsystem`, tutaj `plant.txt`

Równolegle na maszynie węzłów w 4 oddzielnych sesjach należy przygotować polecenia do uruchomienia:

```
student@iot:~$ cd node1
student@iot:~/node1$ /usr/local/bin/EBSimUnoEthCurses -ip 192.168.56.104 node.ino.hex
student@iot:~$ cd node2
student@iot:~/node2$ /usr/local/bin/EBSimUnoEthCurses -ip 192.168.56.104 node.ino.hex
student@iot:~$ cd node3
student@iot:~/node3$ /usr/local/bin/EBSimUnoEthCurses -ip 192.168.56.104 node.ino.hex
student@iot:~$ cd node4
student@iot:~/node4$ /usr/local/bin/EBSimUnoEthCurses -ip 192.168.56.104 node.ino.hex
```

Rysunek 3: Przygotowane polecenia do uruchomienia na 4 sesjach maszyny węzłów - każdy folder oznacza osobny węzeł

Maszyna węzłów wykorzystuje istniejący już na niej emulator z laboratoriów przedmiotu PSIR, dzięki któremu można wykorzystać plik `.hex` emulujący płytkę Arduino.

Widok z maszyny węzłów:

```
GPIO
Z0:0x0000 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x0019 Z5:0x0064
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Start wezła IoT...
Odczytano Instance ID z wejścia Z0: 0
IP wezła: 192.168.56.104
SUKCES: Nasłuchuje na porcie UDP: 10000
Moje ID wezła: 22345
DEBUG: Proba odczytu Z4/Z5 w loop...
DEBUG: Odczytano Origin: 25, 100
Wysyłam REGISTER...
Otrzymano ACK! Jestem zarejestrowany.

GPIO
Z0:0x0001 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x03ff Z5:0x03ff
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Start wezła IoT...
Odczytano Instance ID z wejścia Z0: 1
IP wezła: 192.168.56.104
SUKCES: Nasłuchuje na porcie UDP: 10100
Moje ID wezła: 23222
Wysyłam REGISTER...
Otrzymano ACK! Jestem zarejestrowany.

GPIO
Z0:0x0002 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x03ff Z5:0x03ff
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Start wezła IoT...
Odczytano Instance ID z wejścia Z0: 2
IP wezła: 192.168.56.104
SUKCES: Nasłuchuje na porcie UDP: 10200
Moje ID wezła: 24099
Wysyłam REGISTER...
Otrzymano ACK! Jestem zarejestrowany.

GPIO
Z0:0x0003 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x03ff Z5:0x03ff
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Start wezła IoT...
Odczytano Instance ID z wejścia Z0: 3
IP wezła: 192.168.56.104
SUKCES: Nasłuchuje na porcie UDP: 10300
Moje ID wezła: 24976
Wysyłam REGISTER...
Otrzymano ACK! Jestem zarejestrowany.
```

Rysunek 4: Etap rejestracji węzłów - nawiązanie komunikacji z serwerem, każdy węzeł można odróżnić za pomocą Instance ID (0,1,2,3)

Widok z serwera:

```
Oczekiwanie na węzły (Ctrl+C aby przerwać)...  
Otrzymano User Origin od węzła 22345: 25, 100  
Otrzymano REGISTER od NodeID: 22345 (IP: 192.168.56.104:10000)  
Zarejestrowano nowy węzeł ID: 22345  
Oczekiwanie na węzły (1/4)...  
Otrzymano REGISTER od NodeID: 23222 (IP: 192.168.56.104:10100)  
Zarejestrowano nowy węzeł ID: 23222  
Oczekiwanie na węzły (2/4)...  
Otrzymano REGISTER od NodeID: 24099 (IP: 192.168.56.104:10200)  
Zarejestrowano nowy węzeł ID: 24099  
Oczekiwanie na węzły (3/4)...  
Otrzymano REGISTER od NodeID: 24976 (IP: 192.168.56.104:10300)  
Zarejestrowano nowy węzeł ID: 24976  
Zebrano 4 węzłów. Sortowanie po ID i rozpoczynanie...
```

Rysunek 5: Etap rejestracji węzłów

```
Zebrano 4 węzłów. Sortowanie po ID i rozpoczynanie...  
Auto-adjust: Skala=1.00, Start=(-40.3, 132.1), Zakres X=[-41, 241]  
UZYCIE USER ORIGIN: Start X=25.0, Start Y=100.0  
Partycjonowanie: Ekran X=[0, 200], Pasek=50 px, Krok=5.00  
  
--- PRZYDZIAŁ STREF (Posortowane po ID) ---  
Index 0 -> Węzeł ID 22345: X [0, 50]  
Index 1 -> Węzeł ID 23222: X [50, 100]  
Index 2 -> Węzeł ID 24099: X [100, 150]  
Index 3 -> Węzeł ID 24976: X [150, 200]  
-----  
  
Startujemy od węzła 22345 (Start X: 25.0, Start Y: 100.0)  
Wysyłanie CONFIGU do węzła 22345...
```

Rysunek 6: Przydział stref dla węzłów na podstawie rozmiaru siatki ustawionego na wartość 200 (plik globals.c)

Na początku wywoływana jest złożona funkcja auto-adjust, sortująca węzły po ID (aby porozdzielać obszary rysowania zależnie od ID) mierząca wstępnie wymiary rysunku (aby dobrać skalę, krok i dokładne współrzędne), której dokładniejszy opis znajduje się w sekcji Lsystem 2.6. Każdy węzeł dostaje równy przydział - siatka jest podzielona na 4 równe paski po 50px (na osi X), co zostało ujęte w tabeli na Rysunku 5. Następnie serwer wysyła do węzłów informacje niezbędne do rozpoczęcia pracy - CONFIG, między innymi współrzędne początkowe żółwia w osi x i y, kąt początkowy żółwia, kąt obrotu dla różnych operacji, przeskalowaną długość kroku, itd. Po wysłaniu CONFIGU, następuje wysyłanie do węzłów instrukcji danego L-systemu.

Widok z serwera:

```
Startujemy od węzła 22345 (Start X: 25.0, Start Y: 100.0)
Wysyłanie CONFIGU do węzła 22345...
Odebrano pikseli: 100 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
Otrzymano HANDOVER od węzła startowego. Przerzwanie strumienia.
HANDOVER PROCESSED: X=71731 Y=79393 Idx=74
-> Target Node: 23222
Odebrano pikseli: 1100 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 23222 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=120496 Y=54218 Idx=611
-> Target Node: 24099
Odebrano pikseli: 1500 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24099 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=65000 Y=100000 Idx=866
-> Target Node: 23222
Odebrano pikseli: 1700 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 23222 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=121723 Y=105619 Idx=962
-> Target Node: 24099
Odebrano pikseli: 2500 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24099 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=65000 Y=100000 Idx=1388
-> Target Node: 23222
Odebrano pikseli: 2700 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 23222 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=124697 Y=83960 Idx=1483
-> Target Node: 24099
Odebrano pikseli: 3600 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24099 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=171695 Y=53440 Idx=2012
-> Target Node: 24976
Odebrano pikseli: 4100 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24976 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=105000 Y=100000 Idx=2254
-> Target Node: 24099
Odebrano pikseli: 4600 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24099 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=170275 Y=86885 Idx=2504
-> Target Node: 24976
Odebrano pikseli: 5100 (00B: 0) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24976 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=105000 Y=100000 Idx=2776
-> Target Node: 24099
Odebrano pikseli: 6400 (00B: 24) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24099 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=170810 Y=-15579 Idx=3565
-> Target Node: 24976
Odebrano pikseli: 12500 (00B: 3831) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
Odebrano pikseli: 12500 (00B: 3831) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24976 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=105000 Y=100000 Idx=6946
-> Target Node: 24099
Odebrano pikseli: 13000 (00B: 3897) RX HANDOVER inside send_chunk_reliable. Enqueueing and ABORTING chunk.
STOP Handover stream to node 24099 due to HANDOVER RECEIVED
HANDOVER PROCESSED: X=174591 Y=105891 Idx=7195
-> Target Node: 24976
Odebrano pikseli: 20100 (00B: 8906) RX HANDOVER from Main Loop. Enqueueing.
HANDOVER PROCESSED: X=105000 Y=100000 Idx=11116
-> Target Node: 24099
```

Rysunek 7: Praca węzłów - widoczny mechanizm handover

Logi widoczne na Rysunku 6 pokazują wędrówkę żółwia przez poszczególne strefy odpowiedzialności węzłów. Żółw zaczyna od skrajnej lewej strony, a komunikat RX HANDOVER inside send_chunk_reliable oznacza, że podczas gdy serwer próbował przesłać kolejny fragment instrukcji do węzła, otrzymał od niego informację, że wirtualny żółw opuścił już przypisany mu region. Serwer natychmiast przerywa aktualną transmisję (ABORT), aby nie marnować pasma sieciowego na wysyłanie danych do węzła, który i tak nie może ich już przetworzyć. W logach widzimy procesy przekazywania pracy z węzła 22345 do 23222, następnie do 24099 i dalej.

Ważne jest też to, że:

- Praca nie zawsze przesuwana się w jednym kierunku. Występują powroty (np. z węzła 24099 z powrotem do 23222), co jest naturalne dla fraktali, w których żółw wykonuje nawroty. Każdy taki proces wiąże się z przeliczeniem współrzędnych i wysłaniem nowej konfiguracji do celu (Target Node).
- Wartości pozycji w logach (np. X=71731, Y=79393) to wartości rzeczywiste pomnożone przez 1000. Przykładowo, wartość X=120496 odpowiada współrzędnej 120.496 na siatce ekranu.

- Wraz z upływem czasu rośnie licznik odebranych pikseli. W końcowej fazie logów pojawia się parametr OOB (Out of Bounds), który gwałtownie rośnie (osiągając wartość 8906). Oznacza to, że interpretowany L-system wygenerował instrukcje rysowania poza obszarem siatki 200 na 200.

Widok z maszyny węzłów:

```

GPIO
Z0:0x0000 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x0019 Z5:0x0064
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Start wezła IoT...
Odczytano Instance ID z wejscia Z0: 0
IP wezła: 192.168.56.104
SUKCES: Nasłuchuje na porcie UDP: 10000
Moje ID wezła: 22345
DEBUG: Proba odczytu Z4/Z5 w loop...
DEBUG: Odczytano Origin: 25, 100
Wysyłam REGISTER...
Otrzymano ACK! Jestem zarejestrowany.
Otrzymano CONFIG: StartXY=( 25000, 100000) Angle=0
INIT: Nowa praca, sp=0
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 0 EXPECTED: 0
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 1 EXPECTED: 1
Rendering len=64
HANDOVER at char 74 stack_depth=2
RX PKT: DATA CHUNK sz=72

```

Rysunek 8: Węzeł 1 - najmniej pracy, ten wycinek ekranu to cała jego praca

```

GPIO
Z0:0x0001 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x03ff Z5:0x03ff
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
HANDOVER at char 962 stack_depth=3
RX PKT: DATA CHUNK sz=72
Otrzymano CONFIG: StartXY=( 65000, 100000) Angle=0
INIT: Handover, odtworzono sp=0
Otrzymano CONFIG: StartXY=( 65000, 100000) Angle=0
INIT: Handover, odtworzono sp=0
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 0 EXPECTED: 0
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 1 EXPECTED: 1
Rendering len=64
HANDOVER at char 1483 stack_depth=2
RX PKT: DATA CHUNK sz=72

```

Rysunek 9: Węzeł 2 - na wycinku ekranu jest tylko końcówka jego pracy

```
GPIO
Z0:0x0002 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x03ff Z5:0x03ff
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Otrzymano CONFIG: StartXY=( 105000, 100000) Angle=33800
INIT: Handover, odtworzono sp=0
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 0 EXPECTED: 0
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 1 EXPECTED: 1
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 2 EXPECTED: 2
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 3 EXPECTED: 3
Rendering len=64
HANDOVER at char 7195 stack_depth=3
RX PKT: DATA CHUNK sz=72
Otrzymano CONFIG: StartXY=( 105000, 100000) Angle=0
INIT: Handover, odtworzono sp=0
Otrzymano CONFIG: StartXY=( 105000, 100000) Angle=0
INIT: Handover, odtworzono sp=0
```

Rysunek 10: Węzeł 3 - na wycinku ekranu jest tylko końcówka jego pracy

W przebiegu pracy węzła można wyróżnić następujące etapy i mechanizmy:

- Przyjęcie konfiguracji startowej - log Otrzymano CONFIG potwierdza nadejście pakietu typu MSG_START_WORK. Węzeł otrzymuje precyzyjne współrzędne StartXY oraz kąt nachylenia Angle. Wartości StartXY, podobnie jak na serwerze, są zapisane w postaci przemnożonej przez 1000. Przykładowo, współrzędna 105000 oznacza pozycję 105.0 na siatce ekranu.
- Odtworzenie stanu żółwia - komunikat INIT Handover informuje o tym, że węzeł nie zaczyna pracy od zera, lecz kontynuuje zadanie rozpoczęte przez inny węzeł. Odtworzenie wartości sp (stack pointer) oznacza przywrócenie stanu stosu nawiasów [], co jest niezbędne do poprawnego rysowania rozgałęzień fraktala po przejęciu pracy.
- Odbiór strumienia instrukcji - logi RX PKT DATA CHUNK dokumentują napływanie fragmentów ciągu L-systemu. Każdy fragment ma rozmiar 72 bajtów, co wynika z sumy nagłówka protokołu oraz 64 znaków instrukcji (takich jak F, G, plus, minus).
- Kontrola kolejności danych - mechanizm CHUNK SEQ oraz EXPECTED służy do zapewnienia niezawodności. Węzeł sprawdza numer sekwencyjny każdego kawałka danych. Jeśli numer odebrany zgadza się z oczekiwanym, węzeł przystępuje do interpretacji znaków, co jest sygnalizowane komunikatem Rendering len=64.
- Wyjście poza region i inicjacja handover'u - komunikat HANDOVER at char 7195 stack_depth=3 oznacza, że podczas interpretacji znaku o indeksie 7195 żółw przekroczył granicę strefy, za którą odpowiada ten węzeł. Węzeł natychmiast przerywa rysowanie i wysyła do serwera raport zawierający aktualną pozycję, kąt oraz głębokość stosu (w tym przypadku 3), aby praca mogła zostać przekazana do sąsiedniego węzła.

```
GPIO
Z0:0x0003 Z1:0x03ff Z2:0x03ff Z3:0x03ff Z4:0x03ff Z5:0x03ff
D0:1 D1:1 D2:1 D3:1 D4:1 D5:1 D6:1 D7:1 D8:1 D9:1 D10:1 D11:1 D12:1 D13:1

UART
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 46 EXPECTED: 46
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 47 EXPECTED: 47
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 48 EXPECTED: 48
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 49 EXPECTED: 49
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 50 EXPECTED: 50
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 51 EXPECTED: 51
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 52 EXPECTED: 52
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 53 EXPECTED: 53
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 54 EXPECTED: 54
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 55 EXPECTED: 55
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 56 EXPECTED: 56
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 57 EXPECTED: 57
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 58 EXPECTED: 58
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 59 EXPECTED: 59
Rendering len=64
RX PKT: DATA CHUNK sz=72
CHUNK SEQ: 60 EXPECTED: 60
Rendering len=64
RX PKT: DATA CHUNK sz=25
CHUNK SEQ: 61 EXPECTED: 61
Rendering len=17
HANDOVER at char 11116 stack_depth=0
```

Rysunek 11: Węzeł 4 - na wycinku ekranu jest tylko końcówka jego pracy

Węzeł 4 jest ostatni, który oblicza rysunek (region obliczania po skrajnej prawej stronie). Ma najwięcej pracy, odbiera prawie 15 tysięcy pikseli z 20 tysięcy, a zakończenie obliczeń widać w linii RX PKT: DATA CHUNK sz=25 oraz Rendering len=17, w której jest wyraźnie mniej bajtów do obliczenia. Gdyby przekazywanie instrukcji L-systemu skończyło się w regionie któregoś węzła, to wystąpiłby tam komunikat END_DATA, tak się jednak nie stało i ze względu na złożoność tego fraktala, pikseli OOB, które wyszły poza siatkę grid 200x200 było aż 8906. Komunikat END_DATA widać natomiast w innych fraktalach dostępnych w projekcie jak na przykład krzywa Levy'iego lub trójkąt Sierpińskiego.

```

-> Target Node: 24976
Odebrano pikseli: 20100 (00B: 8906)   RX HANDOVER from Main Loop. Enqueueing.
HANDOVER PROCESSED: X=105000 Y=100000 Idx=11116
-> Target Node: 24099
.....
BRAK AKTYWNOŚCI PRZEZ 15 SEKUND. ZAKOŃCZENIE PRACY.
Zapisano wynik do output.txt
Zapisano obraz do output.pbm (można otworzyć w przeglądarce obrazów)
student@iot:~/l_system_server$ █

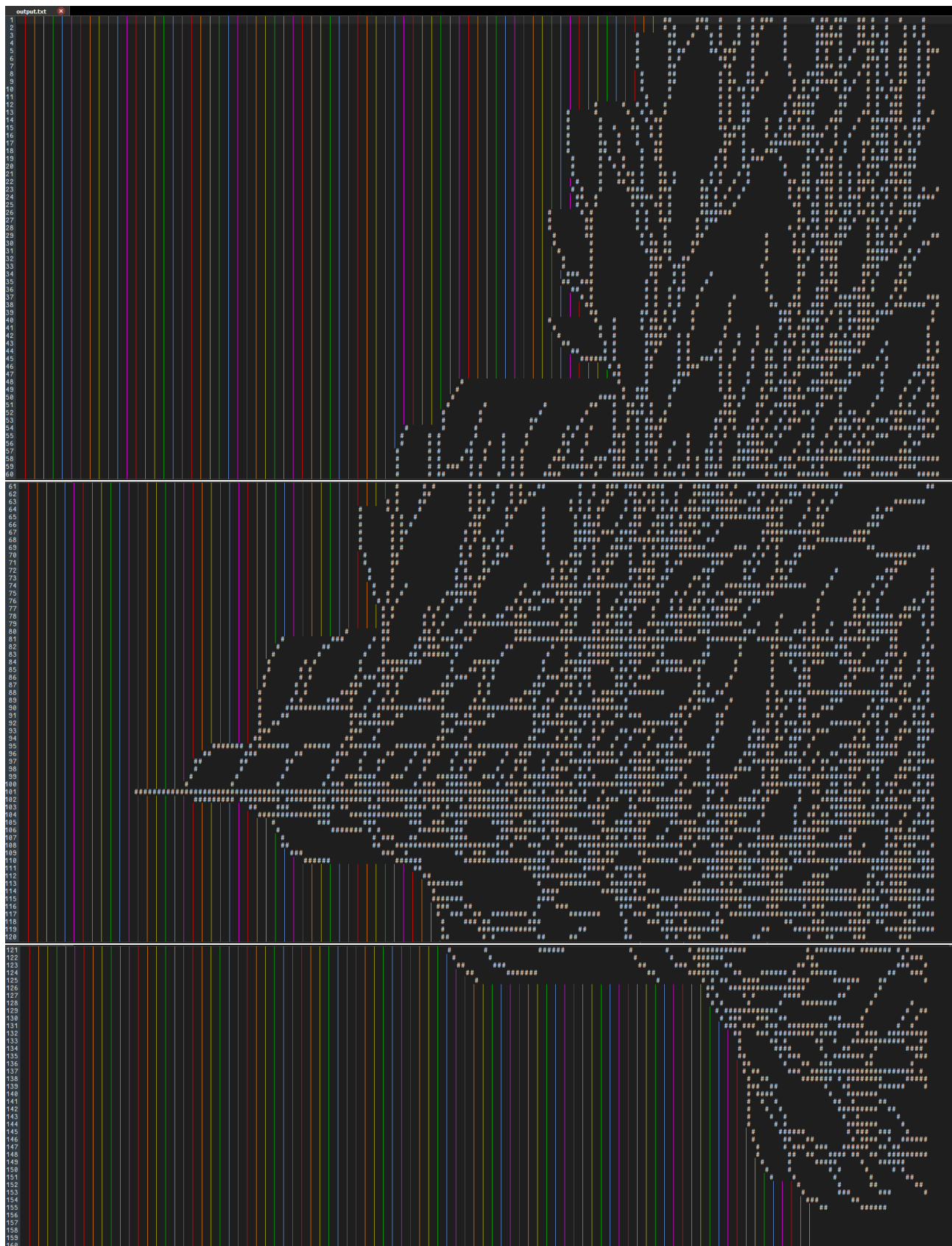
```

Rysunek 12: Koniec pracy węzłów i serwera, zapis do plików

Gdy praca na węzłach jest zakończona, następuje czas nieaktywności, każda sekunda braku jakiegokolwiek zdarzenia to kolejna wypisana kropka na ekranie, gdy uzbiera się 15 kropek (15 sekund), pojawia się komunikat o braku aktywności i zakończeniu pracy. Wynik jest następnie zapisywany do plików .txt i .pbm (portable bitmap), gdzie można zobaczyć obraz odpowiednio w postaci znaczków "#" lub w postaci 0 i 1.



Rysunek 13: Obraz fraktala-rośliny w postaci bitmapy



Rysunek 14: Obraz fraktala-rośliny w postaci tekstowej (znaczk "#")

2.3 Zaprojektowanie i specyfikacja protokołu warstwy aplikacyjnej (ALP)

Komunikacja jest fundamentem rozproszonego systemu, a ponieważ wymagana jest implementacja własnego, binarnego protokołu na UDP, należy go szczegółowo określić, zanim rozpocznie się pisanie kodu serwera (C) i węzłów (Arduino).

Wszystkie struktury danych zdefiniowane w pliku nagłówkowym `alp.h` wykorzystują atrybut `__attribute__((packed))`. Zapobiega to dodawaniu przez kompilator pustych bajtów wyrównujących (padding), zapewniając, że układ danych w pamięci węzła jest identyczny z układem bajtów przesyłanych przez sieć, niezależnie od architektury procesora (np. 8-bit Arduino vs 64-bit Linux Server).

2.3.1 Struktura pakietu

Każdy pakiet ALP rozpoczyna się obowiązkowym nagłówkiem o stałym rozmiarze 4 bajtów, po którym następuje opcjonalny blok danych (payload).

Pole	Typ	Rozmiar	Opis
type	uint8_t	1 bajt	Identyfikator typu wiadomości (określa sposób interpretacji payloadu).
length	uint16_t	2 bajty	Długość danych następujących po nagłówku.
reserved	uint8_t	1 bajt	Pole zarezerwowane (wyrównanie nagłówka do 4 bajtów).

Tabela 1: Struktura nagłówka ALPHeader

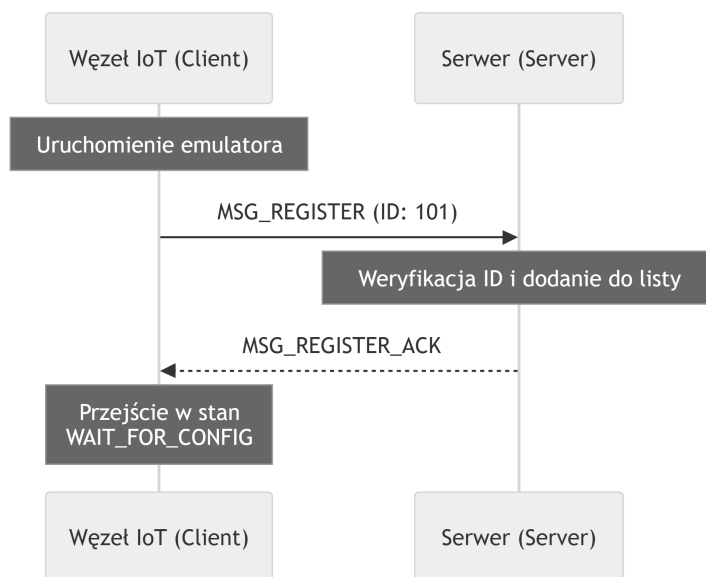
2.3.2 Typy wiadomości i logika sterowania

Protokół definiuje zestaw stałych (od 0x01 do 0x09) sterujących przepływem pracy w systemie klient-serwer.

2.3.3 Rejestracja węzła

MSG_REGISTER (0x01) Wysyłane przez klienta (węzeł). Zawiera strukturę `RegisterPayload` z unikalnym ID węzła oraz preferowanym punktem startowym.

MSG_REGISTER_ACK (0x02) Potwierdzenie wysyłane przez serwer. Sygnalizuje pomyślne dodanie węzła do puli obliczeniowej.



Rysunek 15: Diagram Rejestracji Węzła

2.3.4 Dystrybucja danych (String L-systemu)

Ze względu na ograniczenie MAX_PAYLOAD do 1024 bajtów, długie ciągi znaków L-systemu są fragmentowane.

MSG_DATA_CHUNK (0x07) Serwer przesyła fragment stringa opatrzony numerem sekwencyjnym (ChunkDataHeader).

MSG_CHUNK_ACK (0x08) Klient potwierdza odebranie fragmentu, odsyłając jego numer sekwencyjny. Zapewnia to niezawodność na poziomie aplikacji.

MSG_END_DATA (0x09) Znacznik końca przesyłania danych L-systemu.

2.3.5 Sterowanie pracą (Renderowanie)

MSG_START_WORK (0x03) Rozkaz rozpoczęcia pracy wysyłany przez serwer. Payload WorkConfig zawiera pełny stan żółwia oraz granice przydzielonego obszaru.

MSG_PIXEL_DATA (0x04) Wynik pracy wysyłany przez węzeł. Zawiera współrzędne (x, y) oraz kolor piksela.

2.3.6 Zarządzanie stanem (Handover)

MSG_HANDOVER (0x06) Wysyłane przez węzeł, gdy żółw opuści przydzielony obszar. Payload HandoverPayload zawiera stan żółwia i stosu w momencie przekroczenia granicy, co pozwala serwerowi przekazać zadanie innemu węzłowi.

MSG_WORK_DONE (0x05) Zgłoszenie przez węzeł zakończenia przetwarzania przydzielonego fragmentu.

2.3.7 Szczegóły struktur danych (Payloads)

Poniżej przedstawiono kluczowe pola struktur sterujących logiką obliczeniową.

Tabela 2: Pola struktury WorkConfig (Konfiguracja pracy)

Pole	Typ	Opis
start_x / y	int32_t	Pozycja startowa żółwia.
angle	int32_t	Kąt początkowy.
step	int32_t	Długość kroku (wartość stałoprzecinkowa).
region_min/max	int16_t	Granice obszaru odpowiedzialności węzła.
start_index	uint32_t	Indeks w stringu L-systemu, od którego należy zacząć.
stack... [8]	Tablice	Stan stosu (do 8 poziomów) dla operacji powrotu.

Tabela 3: Pola struktury HandoverPayload (Przekazanie stanu)

Pole	Typ	Opis
current_x / y	int32_t	Pozycja, w której żółw opuścił region.
current_angle	int32_t	Aktualny kąt żółwia.
char_index	uint32_t	Indeks znaku, na którym przerwano pracę.
stack_depth	uint8_t	Aktualna głębokość stosu nawiasów.

2.3.8 Parametry sieciowe

System operuje na następujących portach domyślnych:

- **Port Serwera:** 8080
- **Port Węzła:** 9090
- **Maksymalny rozmiar danych (MTU aplikacji):** 1024 bajty

2.4 Serwer

Serwer jest napisany w języku c i znajduje się na maszynie wirtualnej Linux wykorzystywanej również na laboratoriach przedmiotu PSIR.

2.4.1 Konfiguracja sieciowa serwera i konfiguracja maszyny serwera

Oto jego adresacja:

```
student@iot:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:13:93:ee brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.124/24 brd 192.168.1.255 scope global enp0s3      Statyczny adres używany do zestawienia
                                                                sesji, komunikacji poprzez ssh z serwerem
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe13:93ee/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f6:b3:59 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic enp0s8
        valid_lft 500sec preferred_lft 500sec
    inet6 fe80::a00:27ff:fef6:b359/64 scope link
        valid_lft forever preferred_lft forever
student@iot:~$
```

Rysunek 16: Ważne adresy serwera z komentarzami

Ze względu na przypisany ten sam adres na obu maszynach wirtualnych, zdecydowano się na ręczną konfigurację adresów w pliku /etc/network/interfaces na maszynie:

```
student@iot:~$ cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

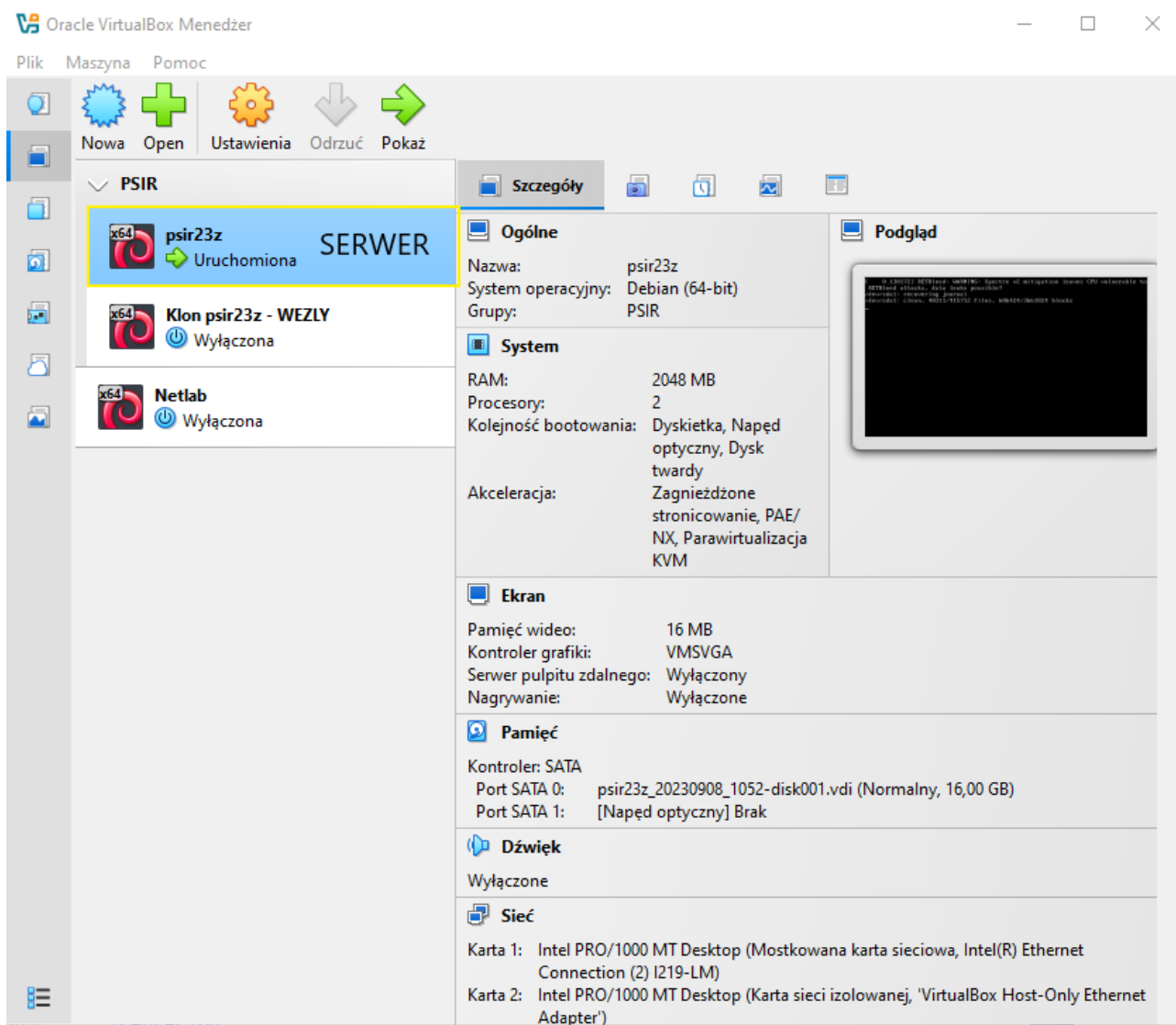
source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

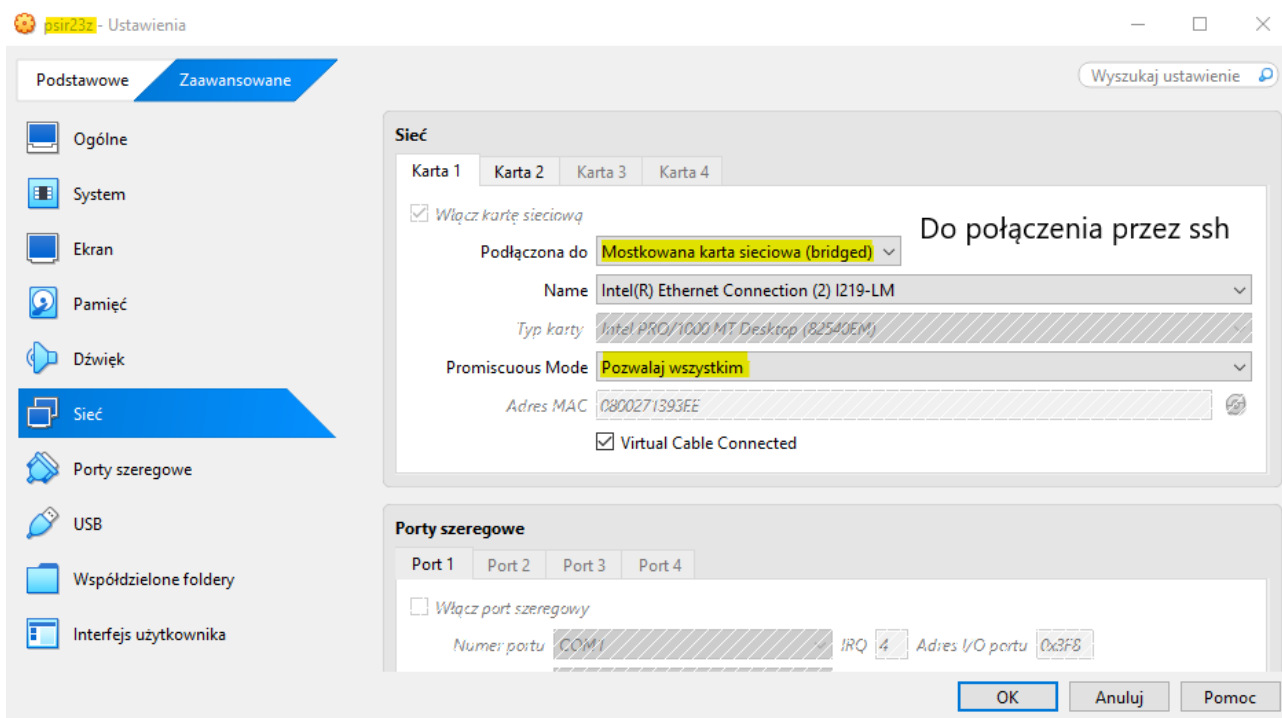
# The primary network interface
allow-hotplug enp0s3
iface enp0s3 inet static
    address 192.168.1.124
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 8.8.8.8 1.1.1.1

allow-hotplug enp0s8
iface enp0s8 inet dhcp
student@iot:~$
```

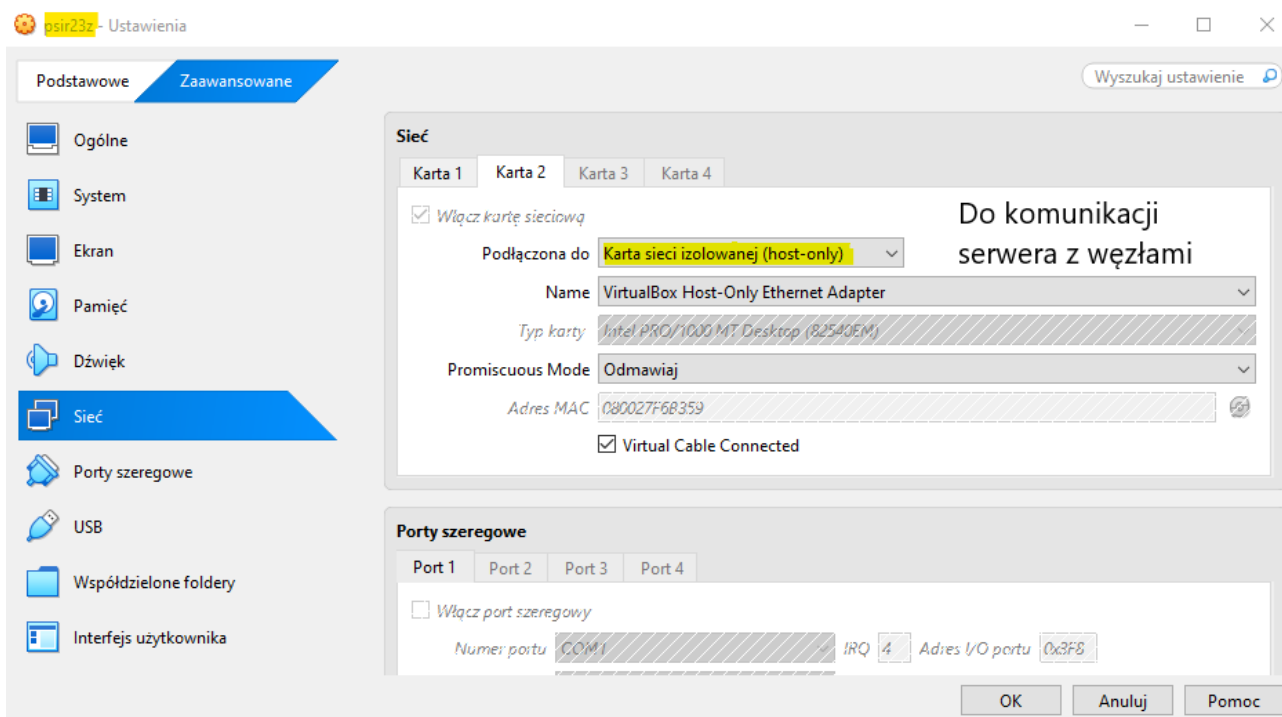
Rysunek 17: Konfiguracja sieciowa w /etc/network/interfaces - adres statyczny do połączenia ssh i dynamiczny do komunikacji serwera z węzłem



Rysunek 18: Widok maszyny serwera w VBox, widać również zasoby maszyny wirtualnej

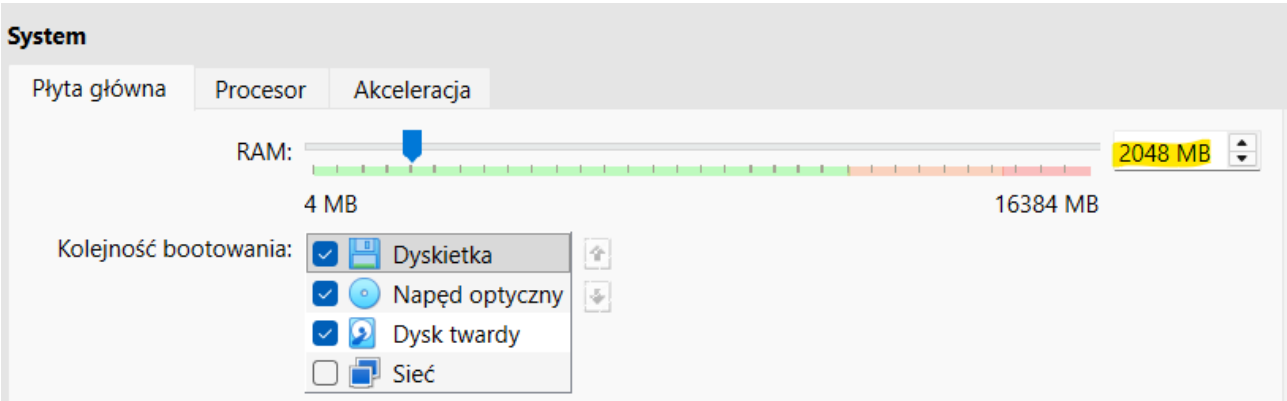


Rysunek 19: Pierwsza karta sieciowa na serwerze do umożliwienia połączenia się przez ssh z hosta

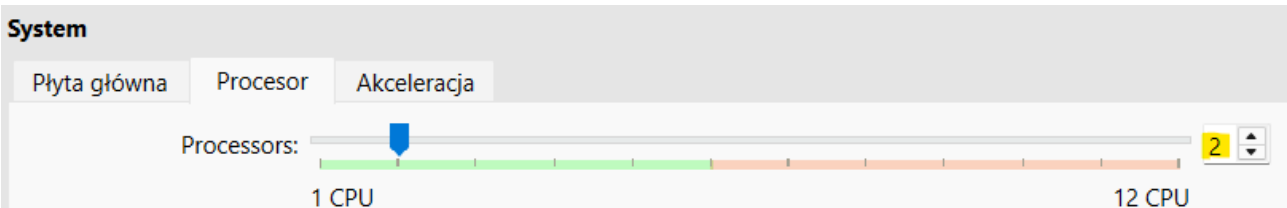


Rysunek 20: Druga karta sieciowa na serwerze do umożliwienia komunikacji serwera z węzłami

Zwiększono zasoby maszyny serwera, bez tego rysunek nie powstanie:



Rysunek 21: Zwiększony RAM do 2GB



Rysunek 22: Zwiększony CPU do 2 CPU

Widok plików serwera:

/home/student/					/home/student/l_system_server/	
Name	Size (KB)	Last modified	Owner	Group	Name	Size (KB)
..					..	
.local					utils.h	1
l_system_server Pliki serwera		2026-01-05...	student	student	utils.c	2
temp - Tu jest emulator		2023-09-08...	student	student	tree.txt	1
.bash_history	1	2023-09-08...	student	student	sierpinski.txt	1
.bash_logout	1	2023-06-22...	student	student	server_new	30
.bashrc	3	2023-06-22...	student	student	server_defs.h	3
.profile	1	2023-06-22...	student	student	plant.txt	1
.Xauthority	1	2026-01-06...	student	student	output.txt	39
					output.pbm	78
					net_core.h	1
					net_core.c	12
					main.c	12
					lsystem.h	1
					lsystem.c	7
					levy.txt	1
					koch_sq.txt	1
					koch.txt	1
					island.txt	1
					globals.c	1
					bush.txt	1
					alp.h	4

Rysunek 23: Struktura katalogów serwera

Pliki serwera są dołączone w plikach projektu, opis tych plików znajduje się na następnej stronie.

2.4.2 server_defs.h – architektura danych i konfiguracja

Plik `server_defs.h` stanowi pomost pomiędzy surowym protokołem komunikacyjnym (`alp.h`), a logiczną implementacją serwera.

- **Konfiguracja Środowiska:** Definiuje stałe `WIDTH` i `HEIGHT` (200x200), które określają rozdzielczość wirtualnego ekranu, na którym renderowany będzie fraktal. Ustalenie stałej `EXPECTED_NODES` na 4 wymusza na serwerze oczekiwanie na konkretną liczbę wykonawców przed rozpoczęciem pracy.
- **Model L-Systemu:** Struktura `LSystem` przechowuje gramatykę fraktala. Zawiera ona tablicę reguł (`Regula`), kąty obrotu oraz flagę `draw_g`.
- **Zarządzanie Węzłami:** Struktura `Node` pozwala serwerowi śledzić stan rozproszonego systemu. Przechowuje ona nie tylko adres sieciowy `sockaddr_in` (niezbędny do komunikacji UDP), ale także status aktywności i unikalne ID węzła.
- **Interfejs Globalny:** Plik ten deklaruje (poprzez słowo kluczowe `extern`) globalne zasoby, takie jak tablica `grid` (pamięć obrazu) oraz licznik odebranych pikseli. Dzięki temu wszystkie moduły serwera operują na tych samych danych, co zapewnia spójność stanu systemu.

2.4.3 main.c – funkcja main, koordynacja w systemie

Plik `main.c` zawiera główną funkcję programu i odpowiada za zarządzanie cyklem życia serwera, koordynację rejestracji węzłów oraz sterowanie procesem renderowania. Jest to punkt wejścia, który łączy logikę sieciową z generowaniem fraktali.

- **Inicjalizacja i przygotowanie:** Na początku program rejestruje procedurę obsługi sygnału `SIGINT` (`Ctrl+C`), co zapewnia bezpieczny zapis wyników w przypadku przerwania pracy. Następnie inicjowane jest gniazdo sieciowe oraz czyszczona jest tablica `grid` reprezentująca wirtualny ekran.
- **Generowanie ciągu L-systemu:** Plik odpowiada za wczytanie konfiguracji z pliku tekstowego i przeprowadzenie iteracyjnego procesu zastępowania znaków zgodnie z regułami. Wynikowy, często bardzo długi łańcuch znaków, jest przechowywany w pamięci serwera przed wysłaniem do węzłów obliczeniowych.
- **Główna pętla zdarzeń:** Serwer pracuje w pętli opartej na funkcji `recvfrom` z ustawionym `timeoutem`. Pozwala to na jednoczesne odbieranie komunikatów od wielu klientów oraz wykrywanie braku aktywności (ciszy w sieci), co służy jako wskaźnik zakończenia pracy całego systemu.
- **Logika partycjonowania i startu:** Po zarejestrowaniu wymaganej liczby węzłów, `main.c` wywołuje algorytm `auto_adjust`, sortuje listę aktywnych węzłów według ich identyfikatorów i dzieli obszar rysowania na pionowe paski (takie strefy odpowiedzialności). Następnie przygotowuje i wysyła pakiety `MSG_START_WORK`.
- **Dystrybucja danych:** Po wysłaniu konfiguracji startowej, plik zarządza procesem niezawodnego przesyłania wygenerowanego ciągu L-systemu do pierwszego węzła w łańcuchu. Wykorzystuje do tego mechanizm dzielenia danych na fragmenty (`chunki`, `chunks`) i obsługuje potencjalne zdarzenia typu `Handover`, które mogą wystąpić już na etapie transmisji danych.

2.4.4 net_core.c – obsługa komunikacji sieciowej i handover'u

Moduł `net_core.c` jest odpowiedzialny za bezpośrednią interakcję z gniazdami systemowymi oraz realizację logiki przesyłu danych zgodnie z regułami zdefiniowanymi w protokole ALP. Plik ten implementuje kluczowe mechanizmy zapewniające stabilność pracy w środowisku opartym na bezpołączeniowym protokole UDP.

Do najważniejszych zadań realizowanych przez ten moduł należą:

- Inicjalizacja serwera - funkcja inicjalizująca tworzy gniazdo UDP i przypisuje je do portu serwera. Serwer zostaje ustawiony w tryb nasłuchiwanie na wszystkich dostępnych interfejsach sieciowych, co umożliwia przyjmowanie zgłoszeń od wielu węzłów obliczeniowych jednocześnie.
- Rejestracja i zarządzanie listą węzłów - moduł przetwarza komunikaty typu `MSG_REGISTER`. Wyodrębnia z nich identyfikator węzła oraz dane adresowe nadawcy, które są następnie zapisywane w tablicy aktywnych jednostek. Serwer obsługuje również aktualizację danych dla węzłów, które zgłaszają się ponownie, i wysyła potwierdzenia rejestracji.

- Niezawodny przesył fragmentów danych - ponieważ protokół UDP nie gwarantuje dostarczenia pakietów, w module zaimplementowano mechanizm potwierdzeń typu Stop-and-Wait. Funkcja `send_chunk_reliable` dzieli ciąg instrukcji L-systemu na mniejsze kawałki, wysyła je do węzła i oczekuje na komunikat ACK. W przypadku braku potwierdzenia lub wystąpienia timeoutu, fragment jest przesyłany ponownie aż do skutku lub osiągnięcia limitu prób.
- Zarządzanie kolejką przekazywania pracy - w systemie rozproszonym komunikaty o zmianie regionu (czyli handover) mogą nadejść w momentach, gdy serwer jest zajęty przesyłaniem danych. Moduł wykorzystuje strukturę kolejki cyklicznej do buforowania tych zdarzeń. Pozwala to na bezpieczne odłożenie informacji o zmianie wykonawcy i przetworzenie jej natychmiast po zakończeniu bieżącej operacji sieciowej.
- Logika handover'u - funkcja `perform_handover` realizuje proces przenoszenia obliczeń między węzłami. Na podstawie aktualnej pozycji wirtualnego żółwia oblicza ona indeks węzła docelowego, który powinien kontynuować rysowanie. Następnie konstruuje i wysyła pakiet konfiguracyjny zawierający dokładny stan pracy (pozycja, kąt, głębokość stosu) oraz inicjuje przesyłanie pozostałej części ciągu instrukcji do nowego wykonawcy.

2.4.5 `Isystem.c` - opis znajduje się w dalszej części raportu [2.6](#)

2.4.6 `utils.c` – przetwarzanie danych wyjściowych i obsługa pikseli

Plik `utils.c` pełni funkcję modułu pomocniczego, który odpowiada za gromadzenie danych spływających od węzłów obliczeniowych oraz ich finalną prezentację. Jest to miejsce, w którym rozproszone wyniki pracy są składane w spójną całość.

Główne funkcje tego modułu to:

- Obsługa przychodzących pikseli - funkcja `handle_pixel` analizuje pakiety zawierające współrzędne oraz znaki ASCII pojedynczych punktów obrazu. Moduł weryfikuje, czy dany punkt mieści się w dopuszczalnym zakresie wirtualnego ekranu, a następnie zapisuje go w głównej macierzy. Dodatkowo realizowane jest zliczanie odebranych danych oraz monitorowanie punktów, które znalazły się poza obszarem rysowania.
- Zapis do formatu tekstowego - funkcja `save_results` generuje plik `output.txt`, będący reprezentacją fraktala w formie grafiki ASCII. Każdy element macierzy jest przenoszony do pliku jako pojedynczy znak, co pozwala na szybki podgląd wyniku pracy w dowolnym edytorze tekstowym.
- Eksport do formatu graficznego - poza plikiem tekstowym, moduł tworzy plik o rozszerzeniu `pbm` (Portable Bitmap). Jest to prosty format graficzny, w którym znaki z macierzy są konwertowane na wartości binarne (0 dla tła, 1 dla narysowanej linii). Dzięki temu wynik pracy serwera może zostać wyświetlony w standardowych przeglądarkach obrazów jako czarno-biała grafika rastrowa.

2.4.7 `globals.c` – definicje zasobów współdzielonych

Plik `globals.c` jest modułem, w którym następuje fizyczna rezerwacja pamięci dla wszystkich kluczowych zmiennych wykorzystywanych przez serwer. W odróżnieniu od plików nagłówkowych, które jedynie informują o istnieniu zmiennych, ten plik tworzy przestrzeń w pamięci operacyjnej komputera.

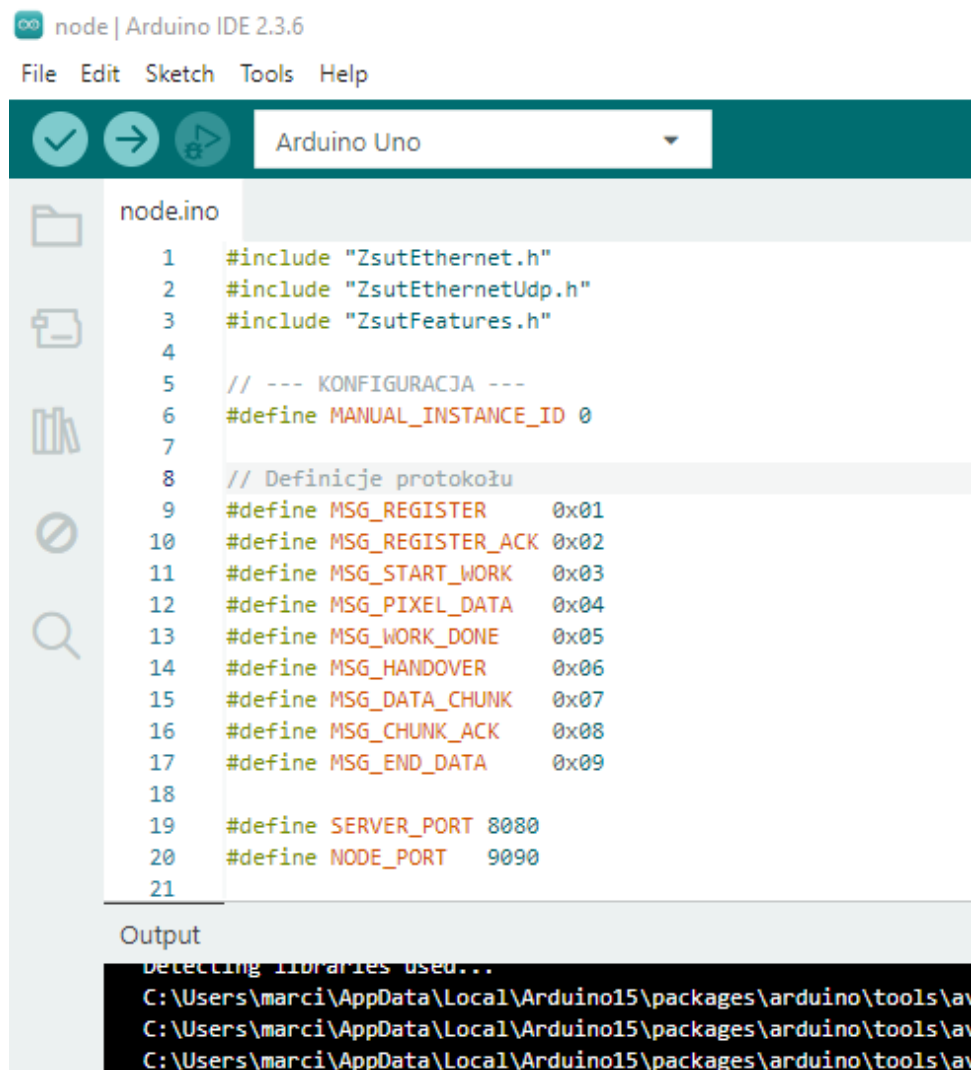
W module tym zdefiniowano:

- Pamięć obrazu - tablica `grid` o rozmiarze 200 na 200 znaków, która stanowi centralny bufor dla renderowanego fraktala. Jest ona dostępna dla wszystkich funkcji zajmujących się odbiorem danych i zapisem wyników.
- Zasoby sieciowe - deskryptor gniazda `sockfd` oraz struktury przechowujące adresy serwera i klientów. Te zmienne są niezbędne do utrzymania komunikacji UDP przez cały czas działania programu.
- Bufory tekstowe - wielokilobajtowy bufor przeznaczony na wygenerowany ciąg instrukcji L-systemu. Rezerwacja stałej, dużej przestrzeni zapobiega problemom z fragmentacją pamięci podczas generowania skomplikowanych fraktali o wielu tysiącach znaków.
- Stan systemu rozproszonego - tablica przechowująca dane wszystkich zarejestrowanych węzłów, liczniki odebranych pikseli oraz zmienne przechowujące parametry geometryczne, takie jak aktualna długość kroku czy szerokość pasów, na które podzielono ekran.

2.5 Węzeł IoT

Węzły napisane są w języku C, mają wspólny kod (powielony wiele razy), otwierany w środowisku Arduino IDE. W przeciwieństwie do serwera, rozszerzenie pliku kodu to nie .c, a .ino, ze względu na środowisko Arduino (trzy ostatnie litery, stąd nazwa), w którym kod jest kompilowany i sprowadzany do postaci .hex - binarnego obrazu firmware'u (lub zwyczajnie skompilowanego kodu maszynowego). Taki kompilat jest przenoszony następnie na maszynę węzłów, która jest sklonowaną maszyną serwera również z laboratorium 2 PSIR.

Ustawienia środowiska Arduino IDE zostały przeprowadzone w całości zgodnie z instrukcją do laboratorium 2 (plik pdf na serwerze Studia3elka) przedmiotu PSIR, z tą drobną różnicą, że bez użycia fizycznej płytki (a więc bez przypisania portów w środowisku).



```
node | Arduino IDE 2.3.6
File Edit Sketch Tools Help

Arduino Uno

node.ino
1  #include "ZsutEthernet.h"
2  #include "ZsutEthernetUdp.h"
3  #include "ZsutFeatures.h"
4
5  // --- KONFIGURACJA ---
6  #define MANUAL_INSTANCE_ID 0
7
8  // Definicje protokołu
9  #define MSG_REGISTER      0x01
10 #define MSG_REGISTER_ACK  0x02
11 #define MSG_START_WORK    0x03
12 #define MSG_PIXEL_DATA    0x04
13 #define MSG_WORK_DONE     0x05
14 #define MSG_HANDOVER      0x06
15 #define MSG_DATA_CHUNK    0x07
16 #define MSG_CHUNK_ACK     0x08
17 #define MSG_END_DATA      0x09
18
19 #define SERVER_PORT 8080
20 #define NODE_PORT   9090
21

Output
detecting libraries used...
C:\Users\marci\AppData\Local\Arduino15\packages\arduino\tools\av
C:\Users\marci\AppData\Local\Arduino15\packages\arduino\tools\av
C:\Users\marci\AppData\Local\Arduino15\packages\arduino\tools\av
```

Rysunek 24: Widok pliku node.ino w Arduino IDE

Przy tworzeniu kodu na komputerze, co jakiś czas należało zamieścić go na maszynach wirtualnych, aby sprawdzić działanie mechanizmów, klikało się zatem Export Compiled Binary (albo Alt+Ctrl+S) w środowisku Arduino IDE i w plikach pojawiał się folder build a wraz z nim pliki .hex:

ownicy > marci > Pobrane > projekt > projekt > nody > node > build > arduino.avr.uno

Nazwa	Data modyfikacji	Typ	Rozmiar
node.ino.eep	07.01.2026 10:54	Plik EEP	1 KB
node.ino.elf	07.01.2026 10:54	Plik ELF	67 KB
node.ino.hex	07.01.2026 10:54	Plik HEX	30 KB
node.ino.with_bootloader.bin	07.01.2026 10:54	Plik BIN	32 KB
node.ino.with_bootloader.hex	07.01.2026 10:54	Plik HEX	30 KB

Rysunek 25: Do sprawdzania wyników wykorzystywany był wsad standardowy (poznany na laboratorium 2 PSIR)

Struktura katalogów i lokalizacja pliku .hex na maszynie wirtualnej węzła:

/home/student/

Name	Size (KB)	Last modified	Owner	Group
..				
node1		2026-01-05...	student	student
node2		2026-01-05...	student	student
node3		2026-01-05...	student	student
node4		2026-01-05...	student	student
temp		2023-09-08...	student	student
.bash_history	1	2026-01-05...	student	student
.bash_logout	1	2023-06-22...	student	student
.bashrc	3	2023-06-22...	student	student
.profile	1	2023-06-22...	student	student
.Xauthority	1	2026-01-06...	student	student

Rysunek 26: Struktura katalogów, każdy folder to inny węzeł (z innym plikiem infile, poza tym mają te same pliki), widok z MobaXterm

/home/student/node1/

Name	Size (KB)	Last modified	Owner	Group
..				
emulino.eeprom	1	2026-01-05...	student	student
infile.txt	1	2026-01-05...	student	student
node.ino.hex	26	2026-01-05...	student	student
outfile.txt	0	2026-01-05...	student	student

Rysunek 27: Lokalizacja pliku .hex w pojedynczym węźle

Mimo że wszystkie węzły znajdują się na jednej maszynie wirtualnej, wystarczy utworzyć 4 oddzielne sesje i w każdej z nich wejść w odpowiedni folder na tej samej maszynie, oraz w każdej sesji wywołać komendę /usr/local/bin/EBSimUnoEthCurses -ip 192.168.56.104 node.ino.hex, aby zasymulować działanie węzłów.

Konfiguracja sieciowa węzła:

```
student@iot:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:13:93:ee brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.122/24 brd 192.168.1.255 scope global enp0s3      Statyczny adres używany do zestawienia sesji,
        valid_lft forever preferred_lft forever                  komunikacji poprzez ssh z maszyną węzłów
    inet6 fe80::a00:27ff:fe13:93ee/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0d:ee:ae brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic enp0s8
        valid_lft 464sec preferred_lft 464sec                  Dynamiczny adres (dhcp) używany do komunikacji
    inet6 fe80::a00:27ff:fe0d:eeae/64 scope link                  między węzłami a serwerem (do zadania), używany w
        valid_lft forever preferred_lft forever                  poleceniu wywołania emulatora (z instrukcji psir lab2)
student@iot:~$
```

Rysunek 28: Adresacja sieciowa maszyny węzłów z opisanymi adresami

Ze względu na przypisany ten sam adres na obu maszynach wirtualnych, zdecydowano się na ręczną konfigurację adresów w pliku `/etc/network/interfaces` na maszynie:

```
student@iot:~$ cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

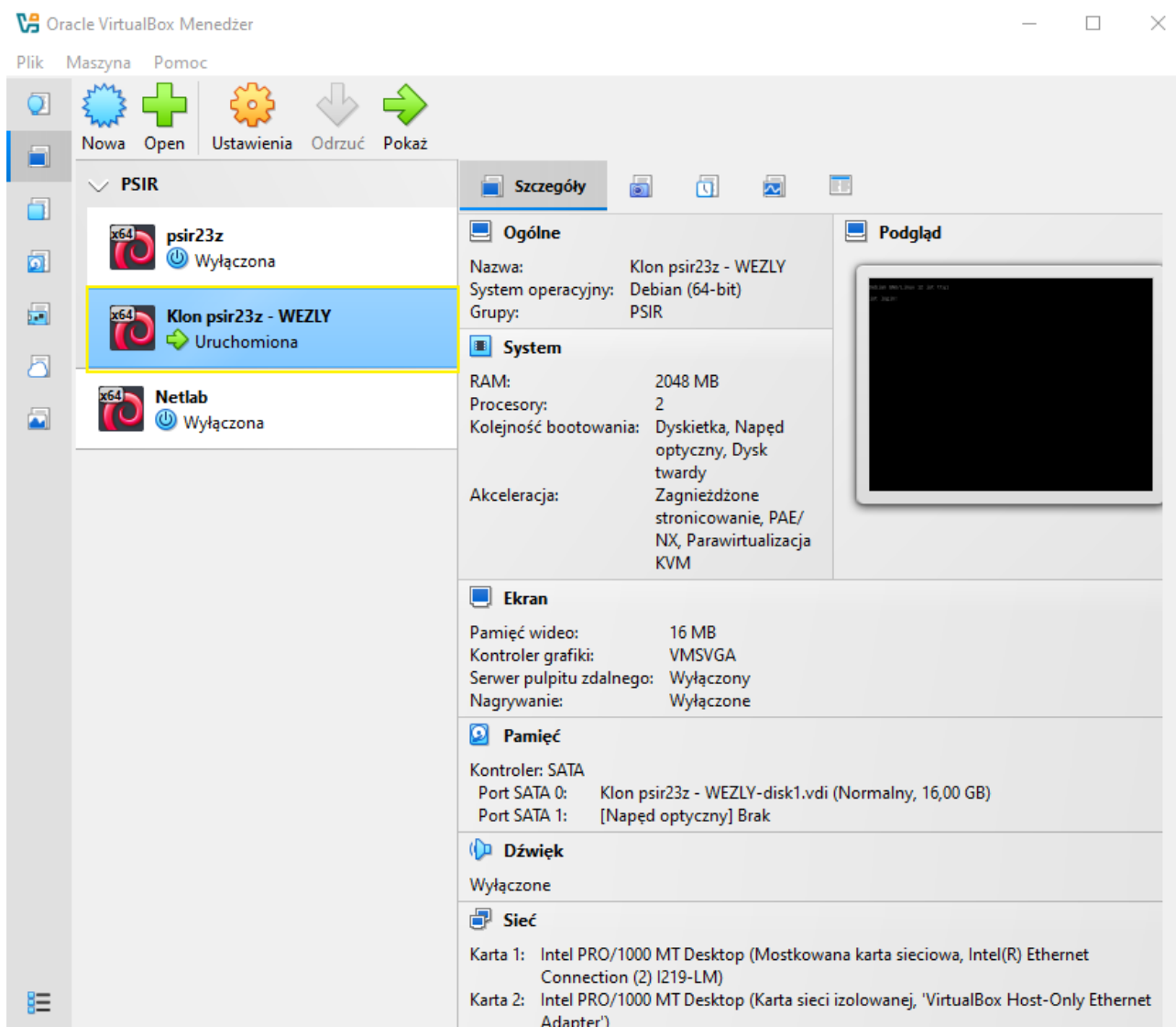
source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

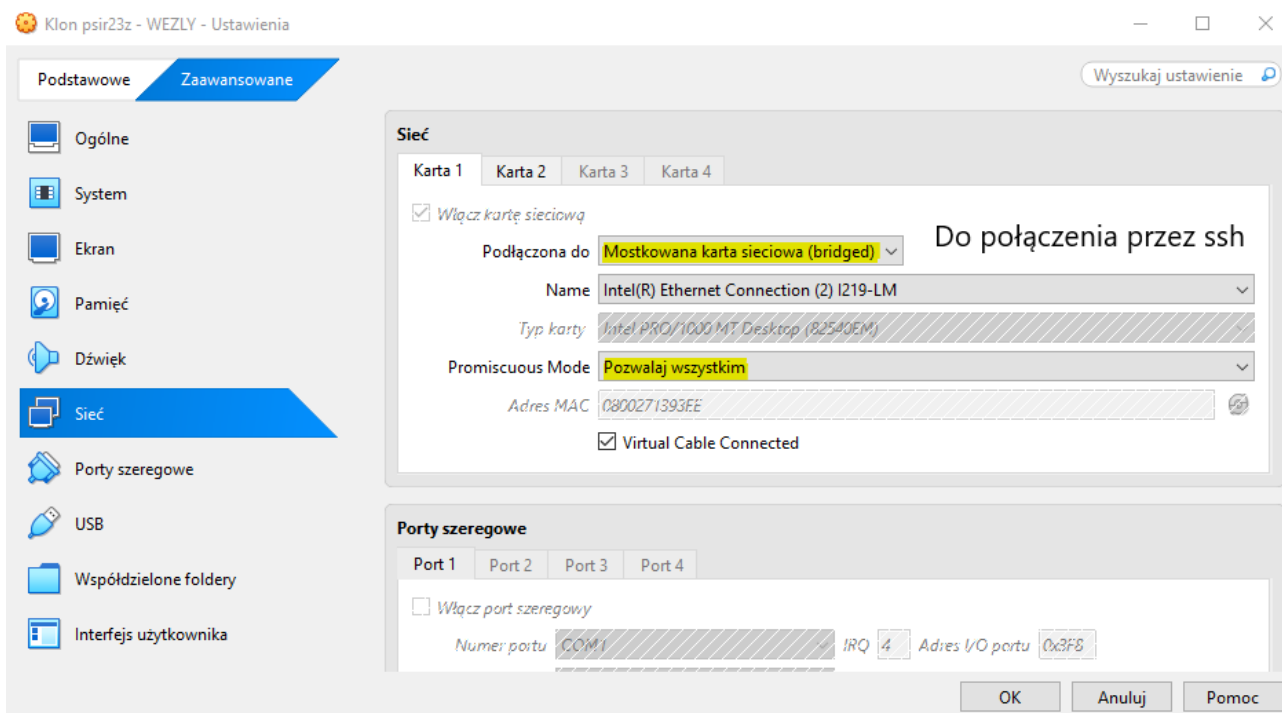
# The primary network interface
allow-hotplug enp0s3
iface enp0s3 inet static
    address 192.168.1.122
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 8.8.8.8 1.1.1.1

allow-hotplug enp0s8
iface enp0s8 inet dhcp
student@iot:~$
```

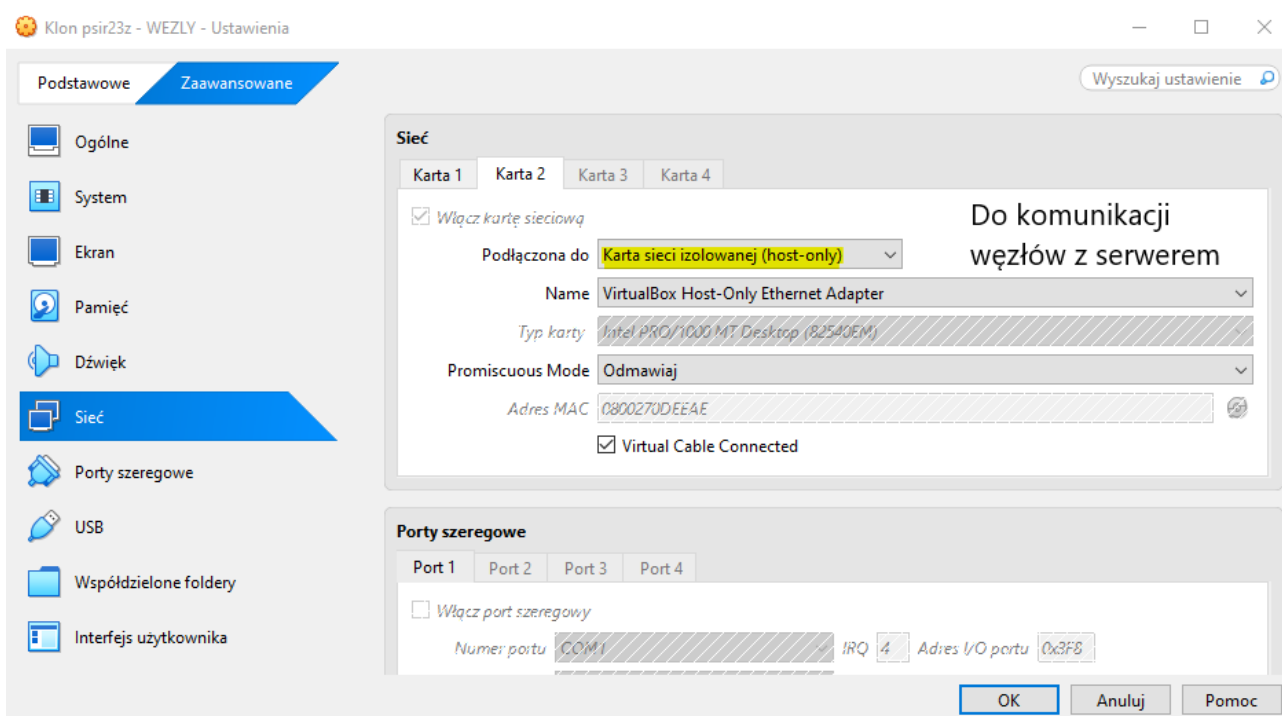
Rysunek 29: Konfiguracja sieciowa w `/etc/network/interfaces` - adres statyczny do połączenia ssh i dynamiczny do komunikacji węzłów z serwerem



Rysunek 30: Widok maszyny węzłów w VBox, widać również zasoby maszyny wirtualnej

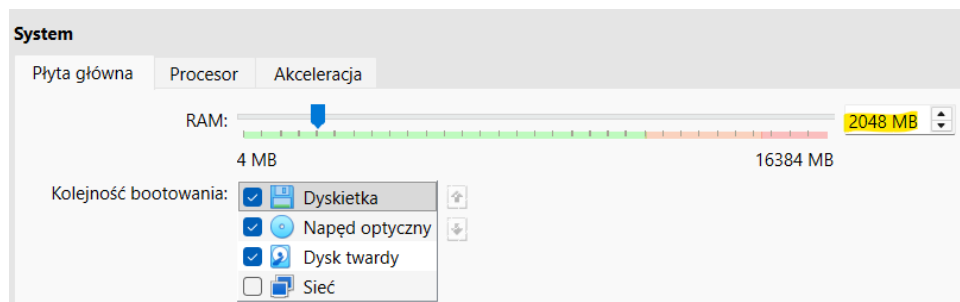


Rysunek 31: Pierwsza karta sieciowa na maszynie węzłów do umożliwienia połączenia się przez ssh z hosta

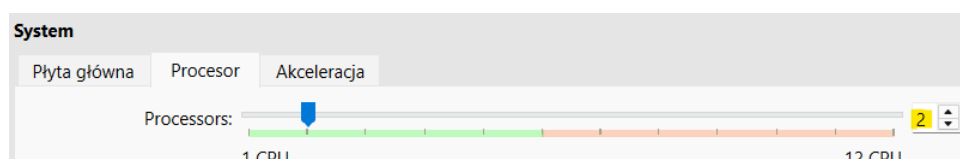


Rysunek 32: Druga karta sieciowa na maszynie węzłów do umożliwienia komunikacji węzłów z serwerem

Zwiększono zasoby maszyny węzłów, bez tego rysunek nie powstanie:



Rysunek 33: Zwiększony RAM do 2GB



Rysunek 34: Zwiększony CPU do 2 CPU

2.6 L-system – logika generowania i interpretacji geometrycznej fraktali

Plik `lsystem.c` - moduł ten odpowiada za przekształcenie teoretycznego opisu L-systemu na konkretny ciąg instrukcji oraz obliczenie parametrów niezbędnych do poprawnego wyświetlenia obrazu na ekranie.

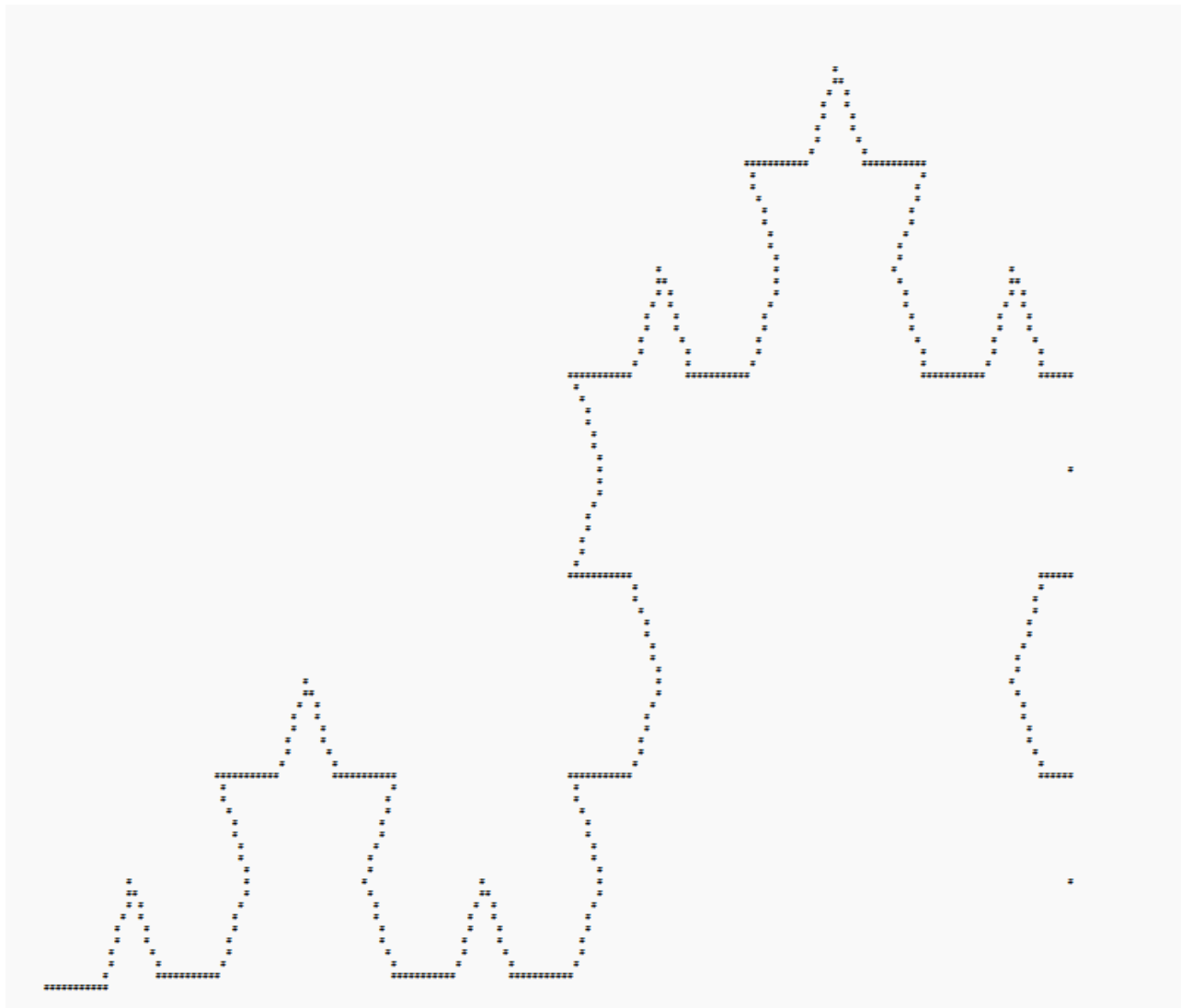
W ramach tego modułu realizowane są następujące zadania:

- Parsowanie plików konfiguracyjnych - funkcja `load_config` umożliwia wczytanie definicji fraktala z zewnętrznego pliku tekstowego. Proces ten obejmuje odczyt aksjomatu, kąta obrotu, liczby iteracji oraz zestawu reguł produkcji. Mechanizm ten zapewnia elastyczność systemu, pozwalając na zmianę rysowanego fraktala bez konieczności rekompilacji kodu źródłowego serwera.
- Iteracyjne generowanie ciągu - funkcja `generuj_ciąg` realizuje proces zastępowania znaków zgodnie ze zdefiniowanymi regułami. W każdej iteracji wejściowy łańcuch znaków jest analizowany znak po znaku, a następnie rozbudowywany o odpowiednie sekwencje instrukcji. Moduł posiada zabezpieczenia ograniczające maksymalną długość ciągu, co chroni pamięć serwera przed przepełnieniem przy bardzo dużej liczbie iteracji.
- Automatyczne skalowanie i centrowanie - najbardziej złożonym elementem pliku jest funkcja `auto_adjust`. Przeprowadza ona pełną symulację rysowania fraktala "na sucho", czyli bez stawiania pikseli w pamięci. Podczas tej symulacji wyznaczone są skrajne współrzędne (minimum i maksimum dla osi X oraz Y), co pozwala obliczyć rzeczywiste wymiary rysunku. Na tej podstawie program wyznacza optymalną skalę (długość kroku) oraz punkt startowy, aby cały fraktal zmieścił się na dostępnym obszarze o wymiarach 200 na 200 jednostek z zachowaniem odpowiednich marginesów.
- Obsługa stosu i rozgałęzień - podczas symulacji geometrii moduł wykorzystuje strukturę stosu do obsługi nawiasów kwadratowych. Pozwala to na poprawne przetwarzanie fraktali posiadających strukturę rozgałęzienia. Zapisywanie i przywracanie stanu żółwia (pozycji oraz kąta) jest kluczowe dla zachowania spójności rysunku, a dane te są później przesyłane w ramach mechanizmu przekazywania pracy między węzłami.
- Zarządzanie parametrami rysowania - moduł kontroluje również specyficzne parametry, takie jak interpretacja znaku G. W zależności od konfiguracji, może on być traktowany jako polecenie narysowania linii lub jedynie przesunięcia wirtualnego żółwia bez pozostawiania śladu, co jest istotne dla poprawnego renderowania różnych typów krzywych fraktalnych.

2.6.1 Dostępne L-systemy

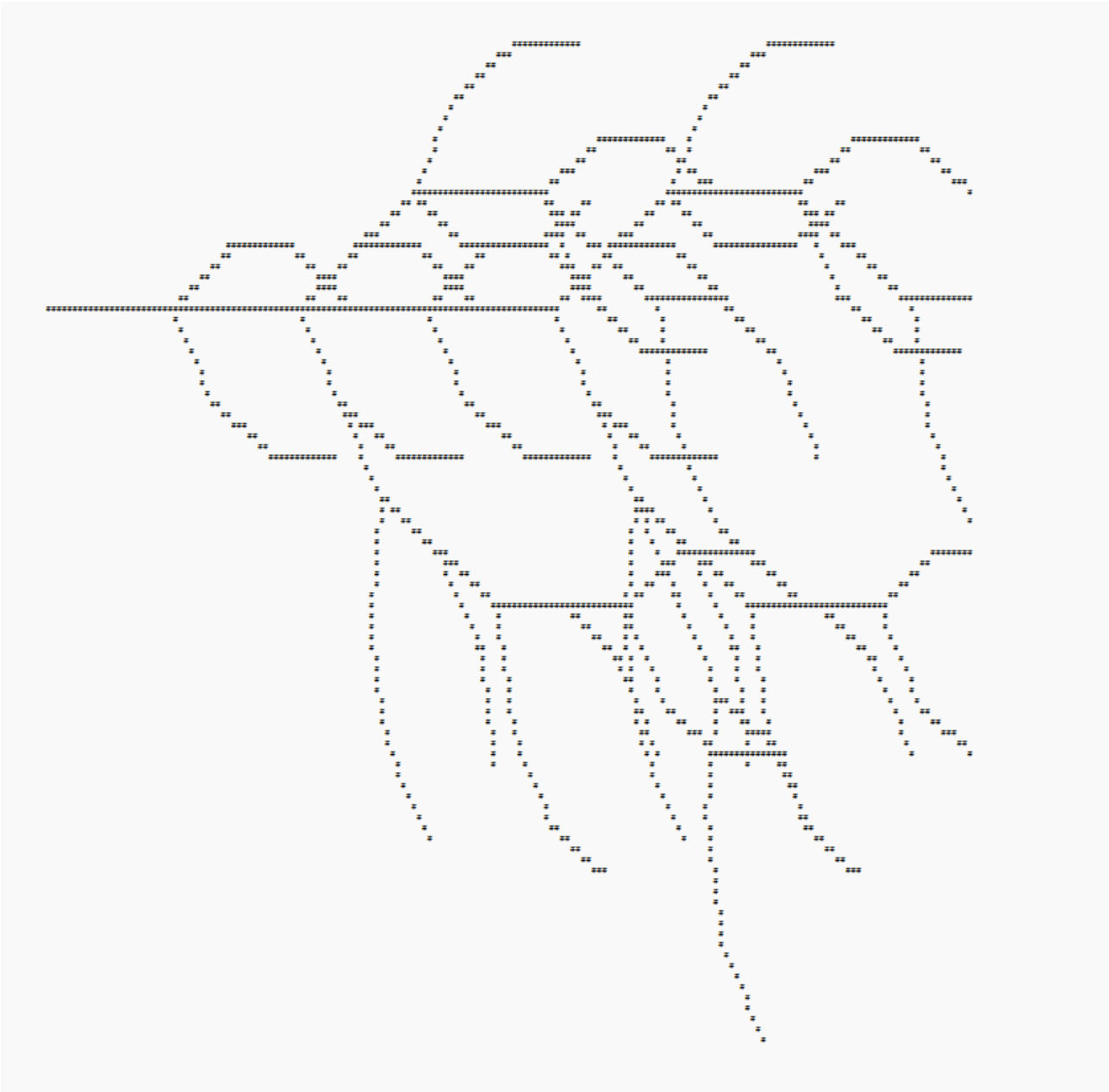
Pliki .txt są dostępne jako załączone do projektu. Różnią się ilością reguł (choć zazwyczaj jest jedna), samymi regułami, aksjomatem (choć zazwyczaj jest to F) i ilością iteracji. Na końcu wyników L-systemów jest opisany pojedynczy plik.txt (rośliny, plant.txt).

Krzywa Kocha:



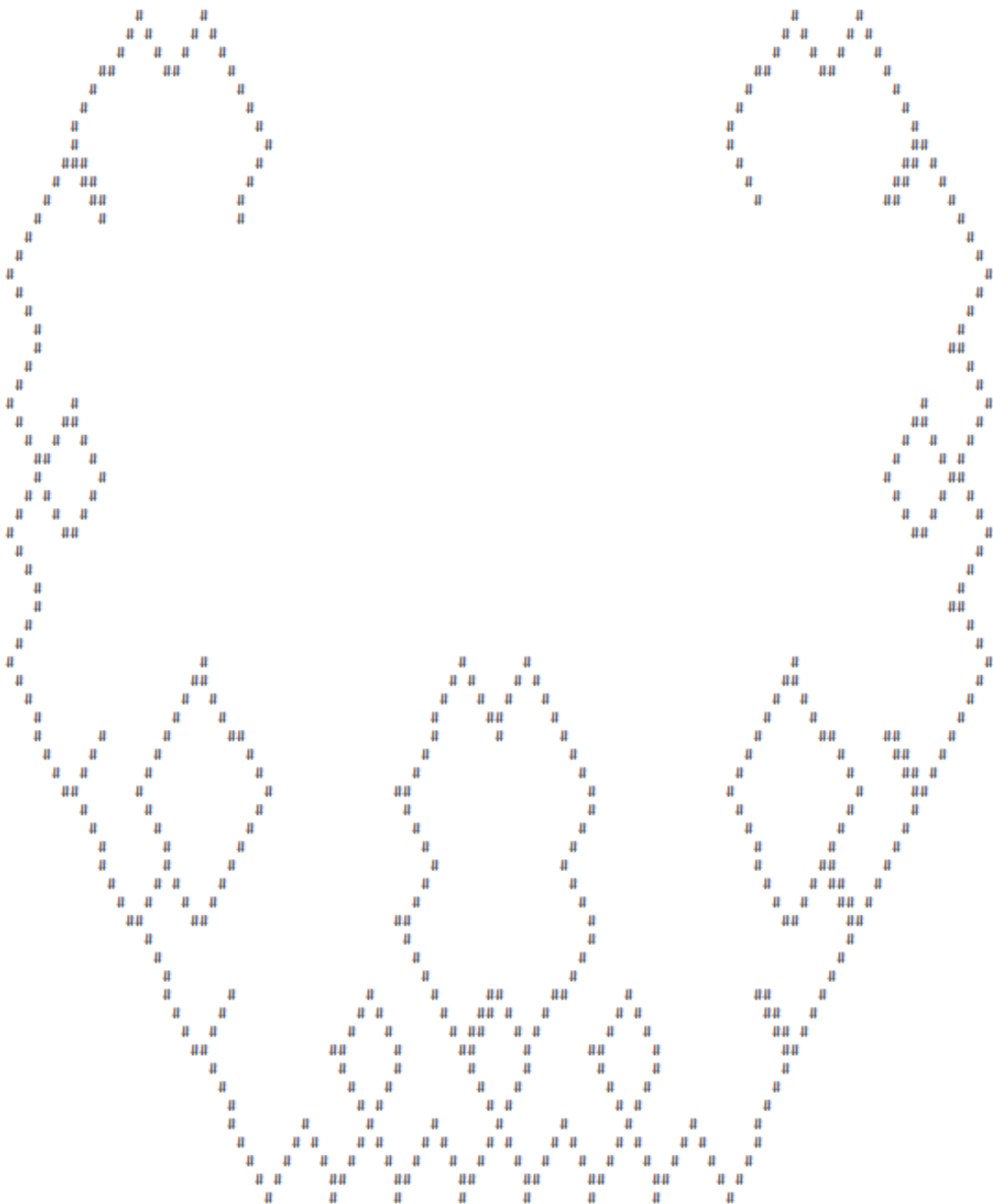
Rysunek 35: Wynik rysowania z pliku koch.txt

Drzewo:



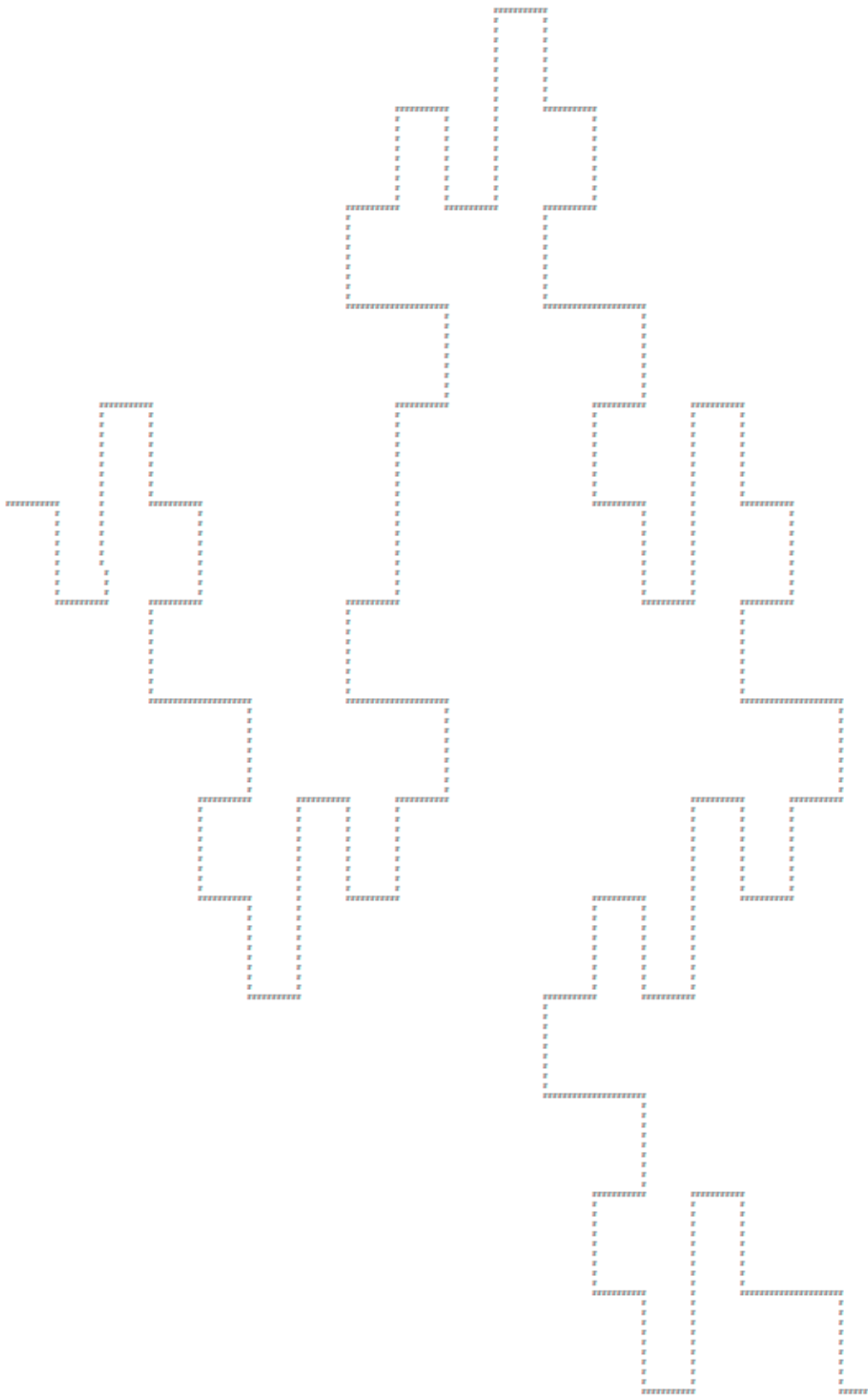
Rysunek 36: Wynik rysowania z pliku tree.txt

Krzywa Levy'iego:



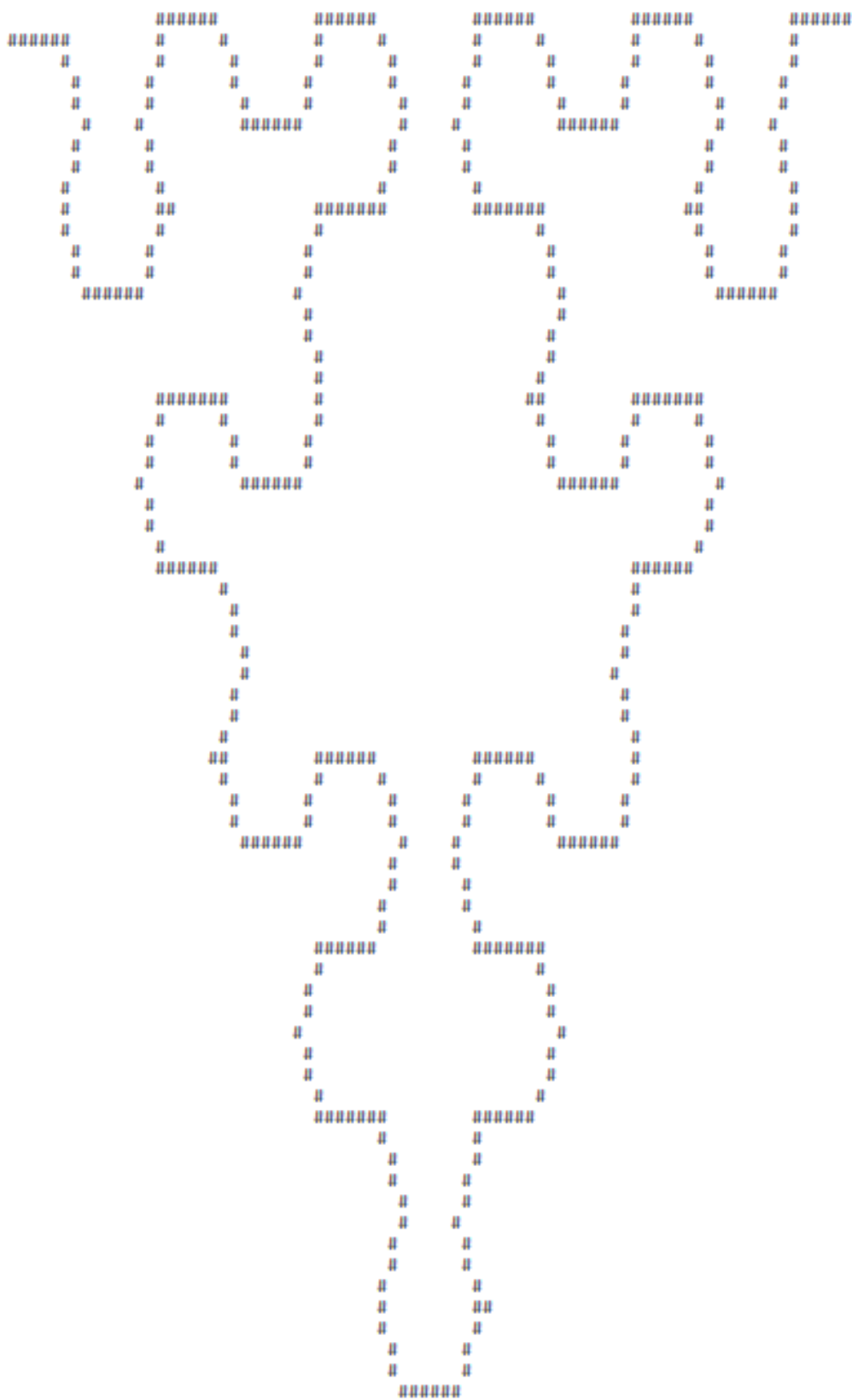
Rysunek 37: Wynik rysowania z pliku levy.txt

Kwadratowa krzywa Kocha:



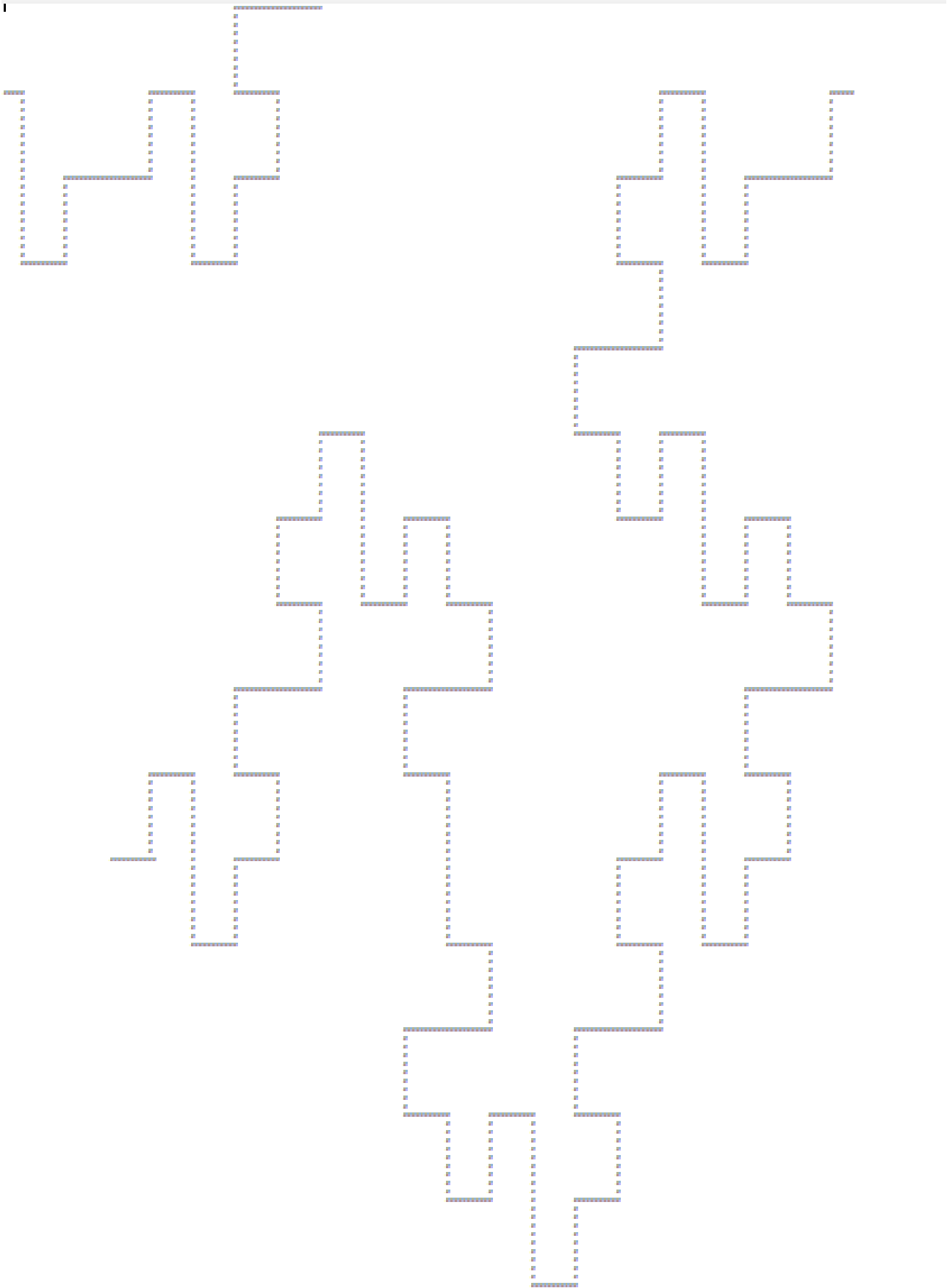
Rysunek 38: Wynik rysowania z pliku koch_sq.txt

Trójkąt Sierpińskiego:



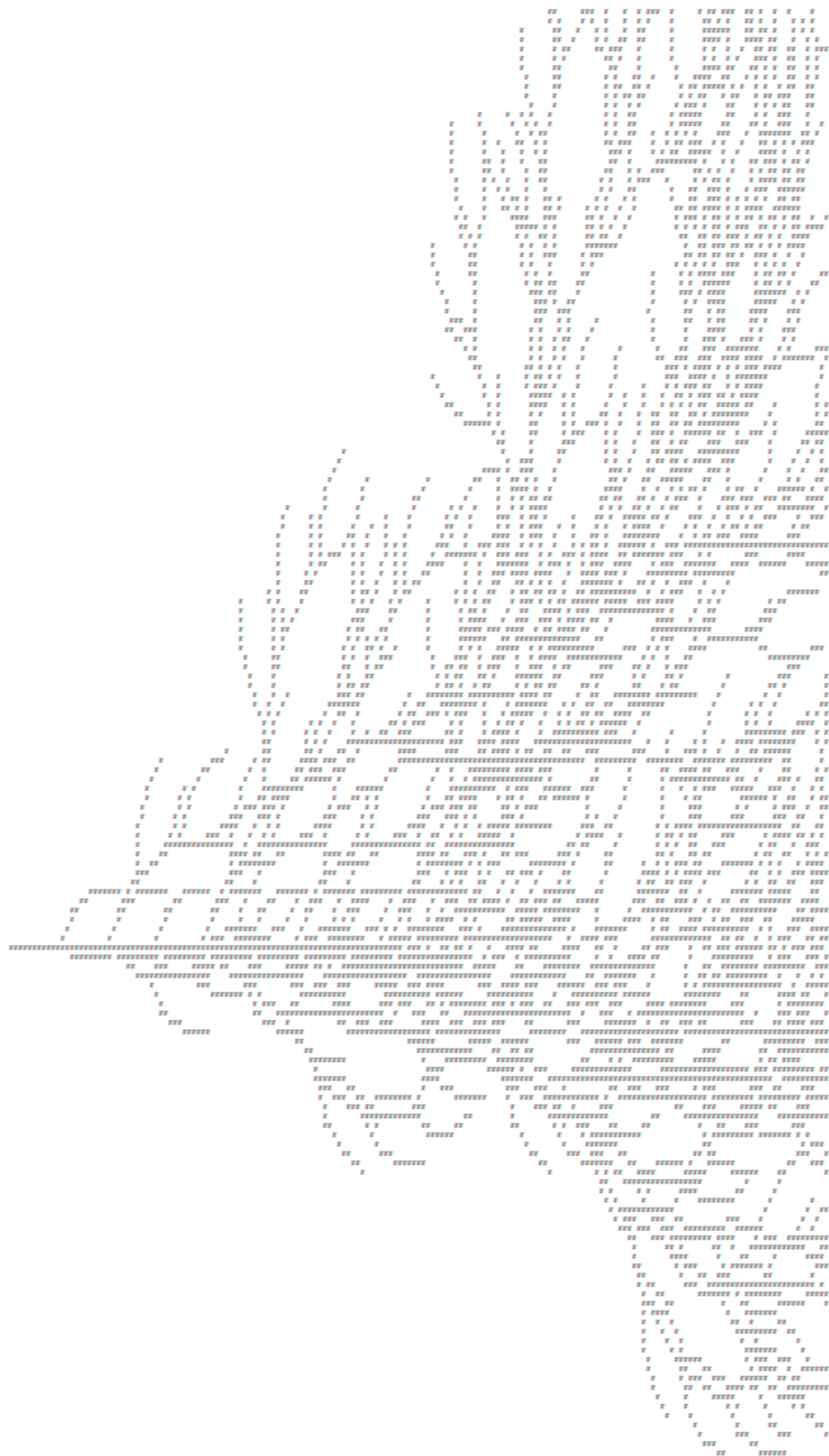
Rysunek 39: Wynik rysowania z pliku sierpinski.txt

Wyspa Kocha:



Rysunek 40: Wynik rysowania z pliku island.txt

Roślina:



Rysunek 41: Wynik rysowania z pliku plant.txt

Krzak:



Rysunek 42: Wynik rysowania z pliku bush.txt

2.6.2 Opis pojedynczego L-systemu w pliku .txt - roślina, plant.txt

W ramach projektu zaimplementowano model rośliny, zdefiniowany w pliku tekstowym `plant.txt`. Poniżej przedstawiono szczegółową analizę parametrów generowania tego fraktala.

```
ITERATIONS: 4
AXIOM: F
ANGLE: 22
START_ANGLE: 0
STEP: 5.0
RULE: F=FF- [-F+F+F] + [+F-F-F]
```

1. Specyfikacja parametrów

Aksjomat (AXIOM): F – punktem początkowym jest pojedynczy odcinek.

Liczba iteracji: 4 – wartość ta pozwala na uzyskanie wysokiego stopnia złożoności struktury bez nadmiernego obciążenia obliczeniowego.

Kąt (ANGLE): 22° – kąt obrotu dobrany w celu naśladowania naturalnego, organicznego wzrostu rośliny. Wartości w przedziale 20° - 25° są uznawane za optymalne do symulacji struktur botanicznych.

Krok (STEP): 5.0 – stała długość pojedynczego segmentu linii.

2. Analiza reguły produkcji

Kluczowym elementem systemu jest reguła: $F \rightarrow FF - [-F+F+F] + [+F-F-F]$. Została ona zaprojektowana w sposób modułowy:

- **Wydłużenie pędu (FF):** Każdy segment F jest zastępowany przez dwa segmenty w linii prostej. Powoduje to, że z każdą iteracją główna oś rośliny rośnie szybciej niż jej odgałęzienia.
- **Mechanizm stosu ([]):** Wykorzystanie nawiasów kwadratowych pozwala na tworzenie rozgałęzień. Program zapisuje bieżący stan (pozycję i kąt) przy znaku $[$, a po narysowaniu gałęzi wraca do niego przy znaku $]$, co umożliwia dalszy wzrost pędu głównego.
- **Struktura liści/korony:** Sekwencje $-F+F+F$ oraz $+F-F-F$ tworzą potrójne rozgałęzienia na końcach bocznych odnóg, co wizualnie symuluje gęstsze ulistnienie.

3. Wnioski i charakterystyka wizualna

Implementacja tego L-systemu pozwala na przetestowanie poprawności działania zagnieżdżonych struktur odgałęzień (za pomocą stosu, zagnieżdżone operacje na stosie). Wynikowy obraz charakteryzuje się:

- Symetrycznym, ale naturalnie wyglądającym rozkładem gałęzi bocznych.
- Wysoką gęstością detali przy relatywnie prostej regule bazowej.

3 Implementacja Mechanizmu Rozproszonego

3.1 Podział Przestrzeni Roboczej

Serwer dzieli wirtualny ekran (canvas) o wymiarach $WIDTH \times HEIGHT$ na pionowe pasy (*slices*). Każdy zarejestrowany węzeł otrzymuje na wyłączność jeden pas przestrzeni (logika w pliku `main.c`). Dzięki zaimplementowanej funkcji `auto_adjust` (plik `lssystem.c`), serwer wstępnie symuluje kształt fraktala bez jego rysowania, aby dobrać optymalną skalę i przesunięcie. Gwarantuje to, że generowany rysunek wypełni dostępny ekran.

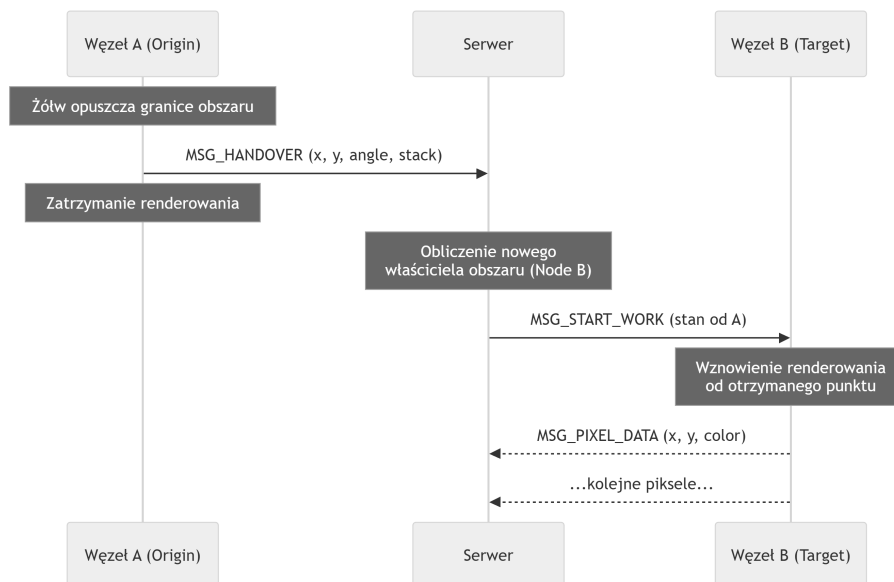
3.2 Mechanizm Handover (Przekazywanie Stanu)

Jest to najważniejszy element systemu, pozwalający na ciągłe rysowanie fraktala przechodzącego przez granice stref węzłów. Proces przebiega następująco:

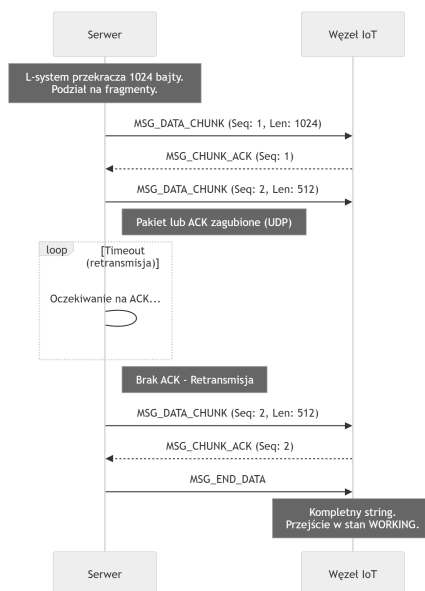
1. **Detekcja:** Węzeł podczas interpretacji ciągu sprawdza, czy nowa pozycja (x, y) mieści się w jego zakresie $[region_min_x, region_max_x]$ (weryfikacja w pliku `node.ino`).
2. **Zrzut Stanu:** Jeśli pozycja wykracza poza zakres, węzeł przerywa pracę i buduje pakiet `HandoverPayload`, zawierający:
 - Aktualne współrzędne (x, y) i kąt.
 - Indeks znaku w ciągu L-systemu, na którym przerwano przetwarzanie.
 - Zawartość stosu (niezbędna dla obsługi rozgałęzień '[' i ']') – zapamiętane pozycje i kąty.
3. **Przekierowanie:** Serwer odbiera pakiet `Handover`, oblicza na podstawie współrzędnej X , który węzeł powinien przejąć pracę, i wysyła do niego komunikat `MSG_START_WORK` ze stanem otrzymanym od poprzednika (funkcja `perform_handover` w `net_core.c`).

3.3 Niezawodność Transmisji

Ze względu na użycie protokołu UDP, który nie gwarantuje dostarczenia pakietów, zaimplementowano programowy mechanizm potwierdzeń dla przesyłania krytycznych danych, jakimi są reguły fraktala. Serwer dzieli długi ciąg (generowany w buforze) na fragmenty po 64 bajty. Każdy fragment musi zostać potwierdzony przez węzeł (`MSG_CHUNK_ACK`) przed wysłaniem kolejnego. W przypadku braku potwierdzenia w określonym czasie (*timeout*), następuje retransmisja.



Rysunek 43: Diagram Procedury Handover



Rysunek 44: Diagram Dystrybucji Danych

3.4 Arytmetyka Stałoprzecinkowa

W celu uniknięcia problemów z różną reprezentacją liczb zmiennoprzecinkowych (`float`) na różnych platformach (PC vs Mikrokontroler/Emulino), kluczowe współrzędne w pakietach konfiguracyjnych są przesyłane jako liczby całkowite (`int32_t`), przeskalowane o czynnik 1000 (*Fixed Point Arithmetic*).

4 Architektura Oprogramowania

4.1 Serwer

Aplikacja serwera napisana w języku C realizuje następujące zadania:

- Wczytywanie definicji L-systemów z plików tekstowych (np. `koch.txt`, `tree.txt`).
- Zarządzanie listą węzłów i kolejką zdarzeń Handover (struktura `ho_queue`).
- Agregacja wyników: odbieranie pikseli i nanoszenie ich na wspólną macierz `grid`.
- Zapis wyniku końcowego do pliku graficznego PGM oraz tekstowego (funkcje w `utils.c`).

4.2 Szczegóły implementacji Węzła (Klient)

Kod węzła został zaimplementowany w pliku `node.ino` i jest przystosowany do działania na emulatorze Arduino z bibliotekami `ZsutEthernet` (do komunikacji UDP) oraz `ZsutFeatures` (do obsługi wejścia/wyjścia). Główna pętla programu `loop()` realizuje maszynę stanów, która steruje zachowaniem węzła w zależności od etapu przetwarzania.

4.2.1 Maszyna Stanów

Logika sterująca węzłem została podzielona na cztery główne stany:

STATE_WAIT_FOR_CONFIG Stan początkowy po uruchomieniu. Węzeł nasłuchuje na porcie UDP pakietów od serwera. W tym stanie węzeł wysyła cyklicznie komunikat `MSG_REGISTER`, dopóki nie otrzyma potwierdzenia rejestracji. Oczekuje na pakiet `MSG_START_WORK`, który zawiera parametry początkowe (pozycję x, y , kąt, reguły L-systemu).

STATE_RECEIVING_DATA Stan aktywowany, gdy ciąg znaków L-systemu jest zbyt długi, by zmieścić się w jednym pakiecie. Węzeł buforuje otrzymywane fragmenty (`MSG_DATA_CHUNK`) i odsyła potwierdzenia (`MSG_CHUNK_ACK`). Dopiero po odebraniu flagi końca danych (`MSG_END_DATA`) następuje przejście do renderowania.

STATE_WORKING Główny stan obliczeniowy. Węzeł iteruje po znakach ciągu L-systemu (stringu) i interpretuje je zgodnie z zasadami grafiki żółwia. W każdej iteracji pętli obliczana jest nowa pozycja. Przed narysowaniem piksela sprawdzany jest warunek graniczny:

$$region_min_x \leq new_x < region_max_x \quad (1)$$

Jeśli warunek jest spełniony, węzeł wysyła `MSG_PIXEL_DATA` do serwera. Jeśli nie – następuje przejście do stanu Handover.

STATE_HANOVER Stan obsługi wyjścia poza obszar. Węzeł pakuje swój aktualny stan (współrzędne, kąt, głębokość stosu, indeks w stringu) do struktury `HandoverPayload` i wysyła komunikat `MSG_HANOVER` do serwera. Następnie węzeł resetuje swoje parametry i wraca do stanu oczekiwania na nowe zadanie.

4.2.2 Interpretacja Grafiki Żółwia (Turtle Graphics)

W stanie `WORKING` znaki są interpretowane w następujący sposób:

- **'F', 'G' (Ruch):** Obliczenie nowych współrzędnych na podstawie trygonometrii:

$$x_{new} = x + step \cdot \cos(\alpha)$$

$$y_{new} = y + step \cdot \sin(\alpha)$$

Dla znaku 'F' wysyłany jest piksel, dla 'G' następuje tylko przesunięcie.

- **'+', '-' (Obrót):** Zmiana aktualnego kąta żółwia o zadaną wartość δ (w lewo lub w prawo).
- **'[' (Stos - push):** Zapisanie aktualnego stanu (x, y, α) na stosie. Jest to niezbędne do rysowania struktur rozgałęzionych (np. drzew).
- **']' (Stos - pop):** Przywrócenie ostatniego zapamiętanego stanu ze stosu, co pozwala żółwiowi "wrócić" do punktu rozgałęzienia.

4.3 Konfiguracja środowiskowa (Pliki infile)

Konfiguracja węzłów nie odbywa się poprzez standardowy odczyt plików tekstowych, lecz poprzez symulację sygnałów na wejściach wirtualnych mikrokontrolera.

Plik `infile.txt` definiuje mapowanie zmiennych symulacyjnych na porty wejściowe oraz ich wartości w czasie. Węzeł w funkcji `setup()` odczytuje te wartości, aby ustalić swoją tożsamość i (w przypadku węzła startowego) pozycję początkową.

Przykładowa zawartość pliku `infile.txt` dla węzła startowego (Origin):

```
1 + nodeId,quantity,Z0
2 + originX,quantity,Z4
3 + originY,quantity,Z5
4 : 0,nodeId,0
5 : 0,originX,25
6 : 0,originY,100
```

Struktura pliku jest interpretowana następująco:

- **Definicje (linie z '+')**: Przypisują nazwy zmiennych do portów emulatora.
 - Z0 służy do przekazania `nodeId` (unikalnego identyfikatora węzła).
 - Z4 i Z5 służą do przekazania współrzędnych startowych `originX` i `originY`.
- **Wartości (linie z ':')**: Określają wartość sygnału w danym czasie (tutaj dla czasu $t = 0$).
 - : 0,nodeId,0 – Ustawia ID węzła na 0.
 - : 0,originX,25 – Ustawia początkową współrzędną X na 25.
 - : 0,originY,100 – Ustawia początkową współrzędną Y na 100.

Dzięki temu mechanizmowi, ten sam kod programu (.hex) wgrany na różne instancje emulatora zachowuje się odmiennie, pobierając unikalne ID i parametry z lokalnego pliku `infile`.